



C / C++

ARDUINO UNO Q

COURSE SLIDES · WSQ

AI-Assisted C Programming for Arduino

WSQ Course Code: TGS-2023039924

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Trainer: Dr Alfred Ang

Version 2.0 · 13 September 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



AI-generated conceptual illustration /not exact hardware

Digital Attendance (Mandatory)

1

TRAQOM / SSG

Use the course-specific digital attendance prompt supplied on the LMS.

2

Assessment attendance

Complete the separate assessment attendance record when directed.

3

Your identity

Use only your own learner record; do not share sign-in details.

4

Trainer support

Ask the trainer if the course/session identity does not match.

About the Trainer



General Trainer

Complete before delivery

NAME

DESIGNATION

QUALIFICATIONS

EXPERTISE

INDUSTRY / TRAINING EXPERIENCE

CONTACT

About the Trainer



Dr Alfred Ang

Trainer / Tertiary Infotech Academy

COURSE

AI-Assisted C Programming for Arduino

FOCUS

Embedded software design and verification

TRAINING APPROACH

Specifications, C logic, progressive activities

CONTACT

enquiry@tertiaryinfotech.com

Learner Introduction

- Your embedded programming experience
- One system you want to build
- One hardware or tooling constraint

Ground Rules

1

Safety

Review voltage, driver and pinout before connecting.

2

Evidence

Record what was run and what remains NOT RUN.

3

AI use

Review every generated patch against contracts and tests.

4

Learning

Use Activity guides for detailed procedures; slides teach mechanisms.

Course Outline

1

Specification, C foundations and safe hardware

K1/A1

2

Modular development and AI-assisted review

K2/A2

3

Controls, memory and data structures

K3/A3

4

Contracts, interoperability and fault evidence

K4/A4

5

User-aligned documentation and bounded extension

K5/A5

Lesson Plan / two days

Time	Day1	Day2
Morning	9-11 Topic1 /2h;11-12 Topic2 /1h	9-12 Topic4 /3h
12:00-1:00	Lunch /1 hour	Lunch /1 hour
Afternoon training	1-3 Topic2 /2h;3-6 Topic3 /3h	1-4 Topic5 /3h
Assessment	No assessment on Day1	WA4-5 PM / PP5-6 PM

WHEN IT MATTERS Approved LU durations2/3/3/3/3h. Day1:8h contact;Day2:6h contact+2h assessment. Lunch excluded; brief rests within contact are not net teaching.

Learning Outcomes

1

LO1

Determine basic software components according to functional and business specifications (K1/A1).

2

LO2

Apply software design methodologies and tools in accordance with organisational practices (K2/A2).

3

LO3

Select controls, elements and features to meet design objectives (K3/A3).

4

LO4

Examine functionality and interoperability of software components (K4/A4).

5

LO5

Generate detailed design documentation mapped to user specifications (K5/A5).

Briefing for Assessment

1

Instrument

Five WA questions and five PP tasks.

2

Preparation

Know requirements, controls, tools, interoperability and docs.

3

Conditions

Open-book and individual; no collaboration on answers.

4

Evidence

Physical board demonstration follows the issued PP instructions.

Assessment & Funding

Written Assessment

- Day2 4:00-5:00 PM
- 60 minutes /5 questions
- K1-K5 / open book

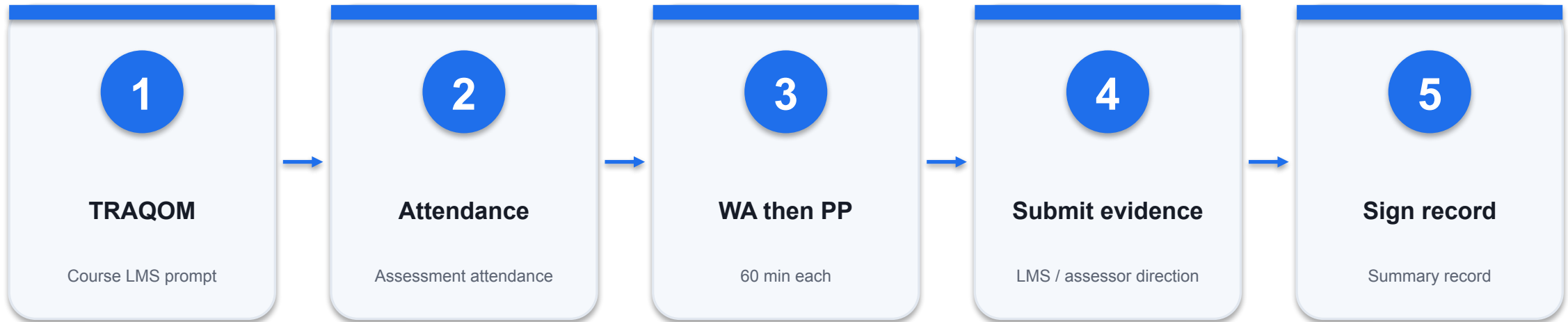
Practical Performance

- Day2 5:00-6:00 PM
- 60 minutes /5 tasks
- A1-A5 / individually

Submission & competence

- Follow issued paper instructions
- All questions/tasks attempted
- Assessor confirms competence

Assessment Flow



ACTUAL PAPERS

Five WA questions K1-K5 and five PP tasks A1-A5. Follow the assessor-issued submission instructions.

Download course material on the LMS

<https://lms-tms.tertiaryinfotech.com/>

My course

AI-Assisted C Programming for Arduino

Course material

Slides / Learner Guide / Activities

Assessment

Written Assessment / Practical Performance

Course-specific access

Use your own learner account and the verified course title.

Course material and assessment areas are distinct. Trainer answers are not learner downloads.

Interface schematic; labels are illustrative, not a live authenticated screenshot.

[Open learner LMS](https://lms-tms.tertiaryinfotech.com/)

Access the Hands-On Activities

Repository

- Learner-safe code and mock data
- Independent Activity folders
- No trainer answer keys

<https://github.com/tertiarycourses/TGS-2023039924-AI-Assisted-C-Programming-for-Arduino>

Get a working copy

- Clone OR download repository ZIP
- Detailed paths in Learner Guide
- Run activities on your own host

Hardware boundary

- Host C tests require no board
- UNO Q upload is a separate check
- 3.3 V-safe wiring review first

Embedded C / a reviewable workbench



Source

Specifications and headers define what must be built.

Tests

Mock inputs establish repeatable acceptance boundaries.

Board

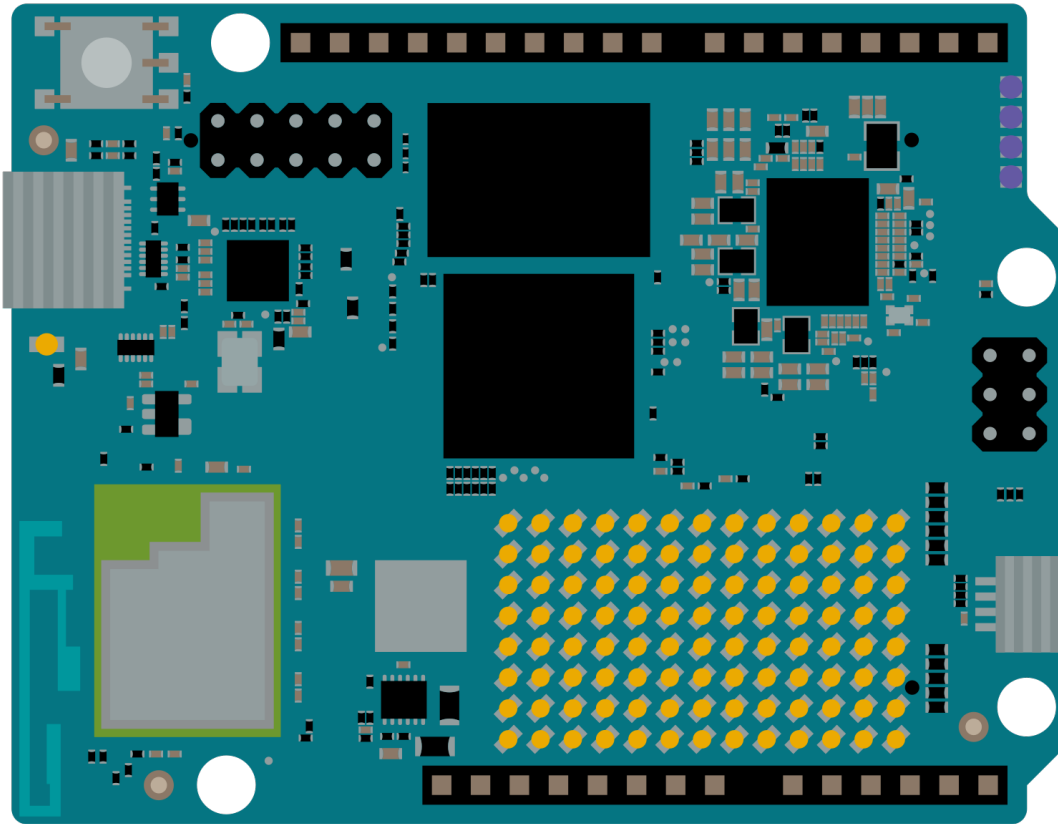
A separate physical review verifies actual wiring and behaviour.

AI

Suggestions remain subject to human and compiler review.

AI-generated conceptual illustration; generic board, not an exact UNO Q hardware reference.

UNO Q / official board illustration



MPU / Linux

QRB2210 runs Debian Linux applications and optional model inference.

MCU / Arduino

STM32U585 runs the sketch and 3.3 V-safe control logic.

Bridge / boundary

Value-based RPC connects the processors; validation stays on the MCU.

Wiring authority

Use June2026 manual; this board image is not a voltage-limit chart.

Official Arduino documentation illustration; see build/source-notes.md. Current June2026 manual wins over older pinout warnings.

TOPIC 1 /K1/A1

Specification, C foundations and safe hardware

Activities 1-3 / specifications, contracts, failure and evidence

Specification becomes a component contract

Exact mechanism / inspect the contract, not just syntax

```
R1 button press -> request outputs  
R2 LED and buzzer share 2000 ms  
R3 loop remains responsive  
input: pressed, now_ms  
state: active, started_ms  
output: Boolean request
```

Input

Separate electrical raw level from pressed intent.

State

Two persistent variables implement a timed interval.

Function

doorbell() has explicit inputs and return value.

Evidence

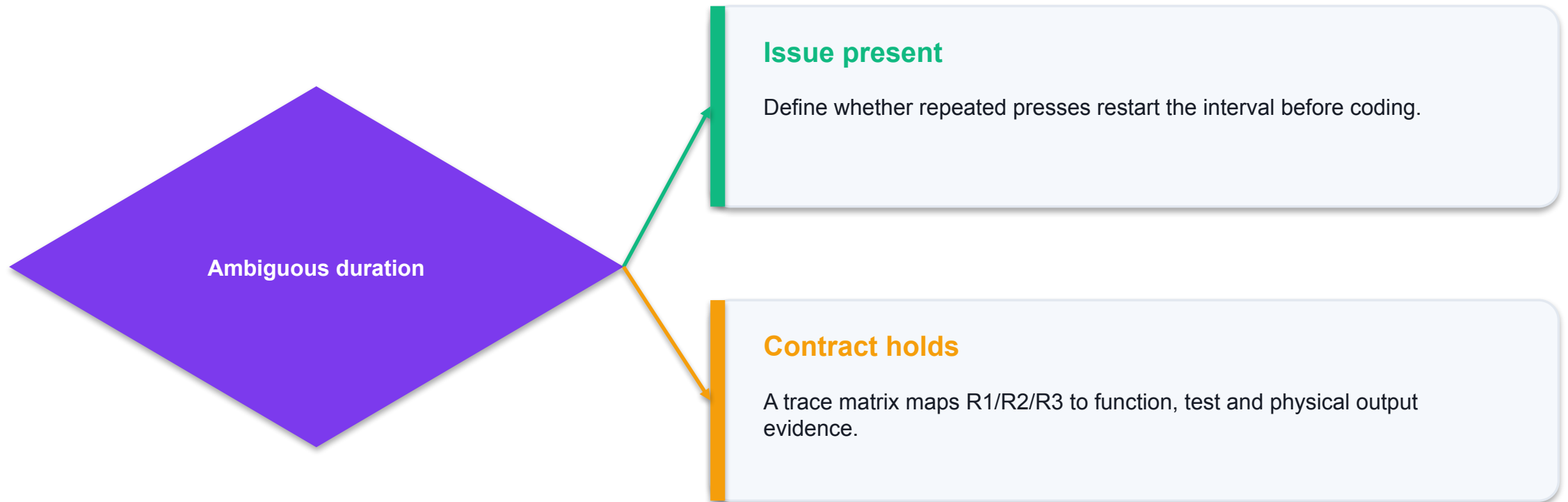
1999/2000 ms tests distinguish the deadline.

Specification becomes a component contract / contract

Contract field	Exact rule
Requirement	Visitor gets visible and audible feedback.
Data type	uint32_t timestamps; int Boolean request.
Function boundary	Pure logic does not configure pins.
Business limit	Buzzer driver rating is a hardware prerequisite.

WHEN IT MATTERS A trace matrix maps R1/R2/R3 to function, test and physical output evidence.

Specification becomes a component contract / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

UNO Q assigns work to two processors

Exact mechanism / inspect the contract, not just syntax

```
Debian Linux / QRB2210
Python app, optional local model
      Bridge RPC
STM32U585 / Arduino sketch
bounded control logic
3.3 V-safe MCU I/O
```

Linux

QRB2210 quad Cortex-A53 up to 2 GHz runs Debian Linux.

MCU

STM32U585 Cortex-M33 up to 160 MHz handles deterministic I/O.

Boundary

Bridge RPC carries explicit values, not shared raw pointers.

Scope

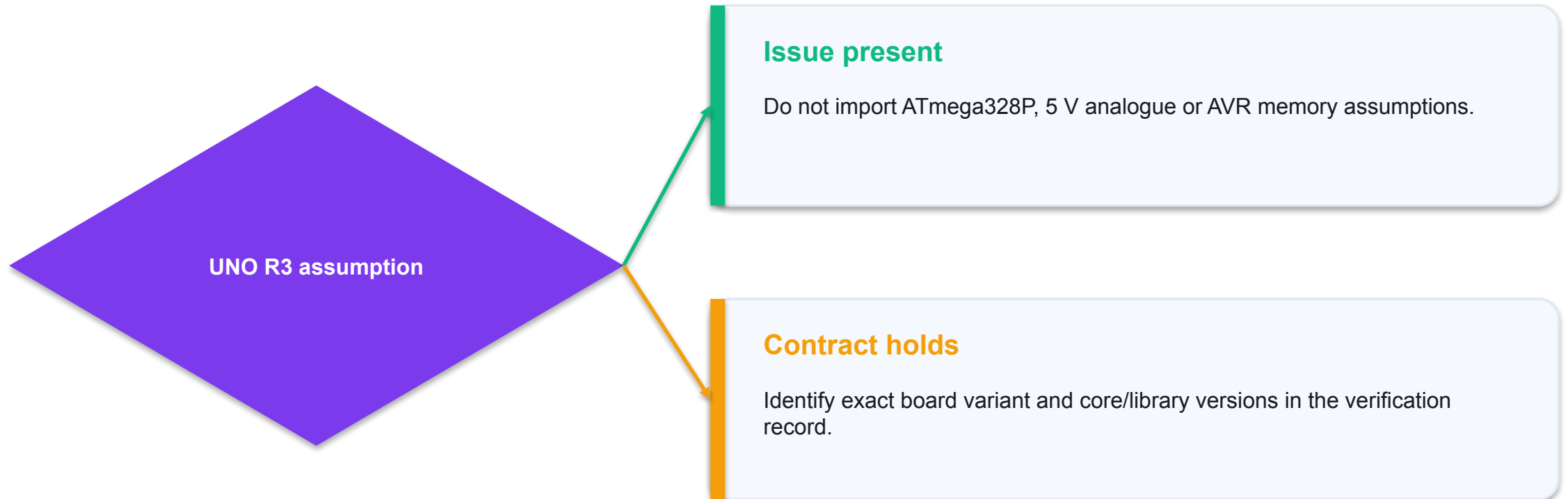
Core activities test C logic; local AI is an optional extension.

UNO Q assigns work to two processors / contract

Contract field	Exact rule
ABX00162	2 GB RAM / 16 GB storage variant.
ABX00173	4 GB RAM / 32 GB storage variant.
MCU memory	Arduino manual states 2 MB flash / 786 kB SRAM.
Authority	Current ABX00162 manual, updated 17 June 2026.

WHEN IT MATTERS Identify exact board variant and core/library versions in the verification record.

UNO Q assigns work to two processors / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Keep analogue inputs inside their range

Exact mechanism / inspect the contract, not just syntax

```
A0..A5 -> STM32 analogue inputs  
valid voltage: GND .. VREF+  
VREF+ approximately 3.3 V  
D3/PB0: maximum 3.6 V  
A4/A5 I2C pullups: 3.3 V only
```

Analogue

A0-A5 are not 5 V-tolerant analogue inputs.

D3

The special PB0 limit forbids a blanket digital 5 V rule.

Pull-ups

External I2C pull-ups must match the approved 3.3 V interface.

Power

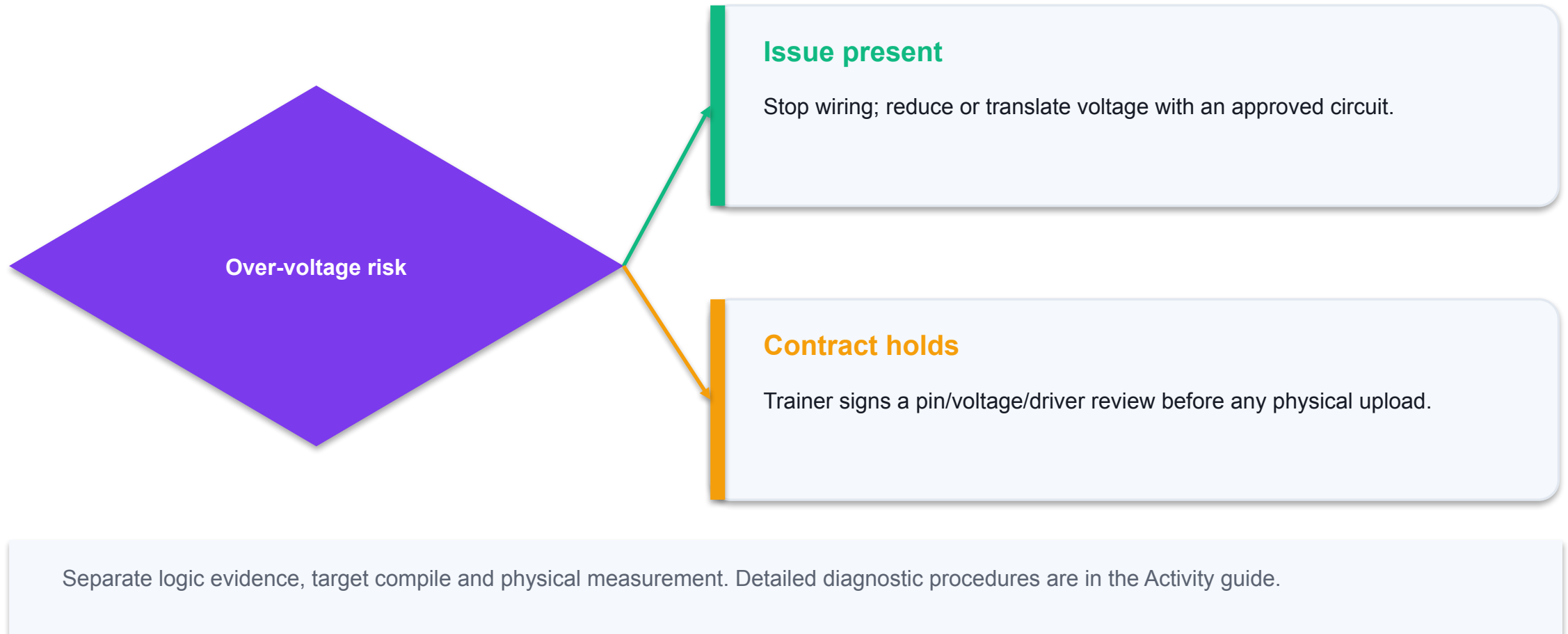
USB PD requests 5 V/3 A; VIN range is 7-24 V.

Keep analogue inputs inside their range / contract

Contract field	Exact rule
Signal review	Verify sensor output level before connecting.
Actuation	Use rated driver; never power a motor from a GPIO.
Qwiic	I2C4 uses PD13 SDA / PD12 SCL at 3.3 V.
Manual evidence	Consult board pinout for exact selected pin.

WHEN IT MATTERS Trainer signs a pin/voltage/driver review before any physical upload.

Keep analogue inputs inside their range / failure



C logic lives behind a C++ sketch

Exact mechanism / inspect the contract, not just syntax

```
#include "logic.h"
void setup() { /* configure I/O */ }
void loop() {
  int request = alarm(25, 30);
  /* adapter writes request */
}
```

Language

An .ino file is compiled as Arduino C++, not pure C.

C module

logic.c remains C-compatible and host-testable.

Header

extern "C" guards preserve linkage when called from C++.

Adapter

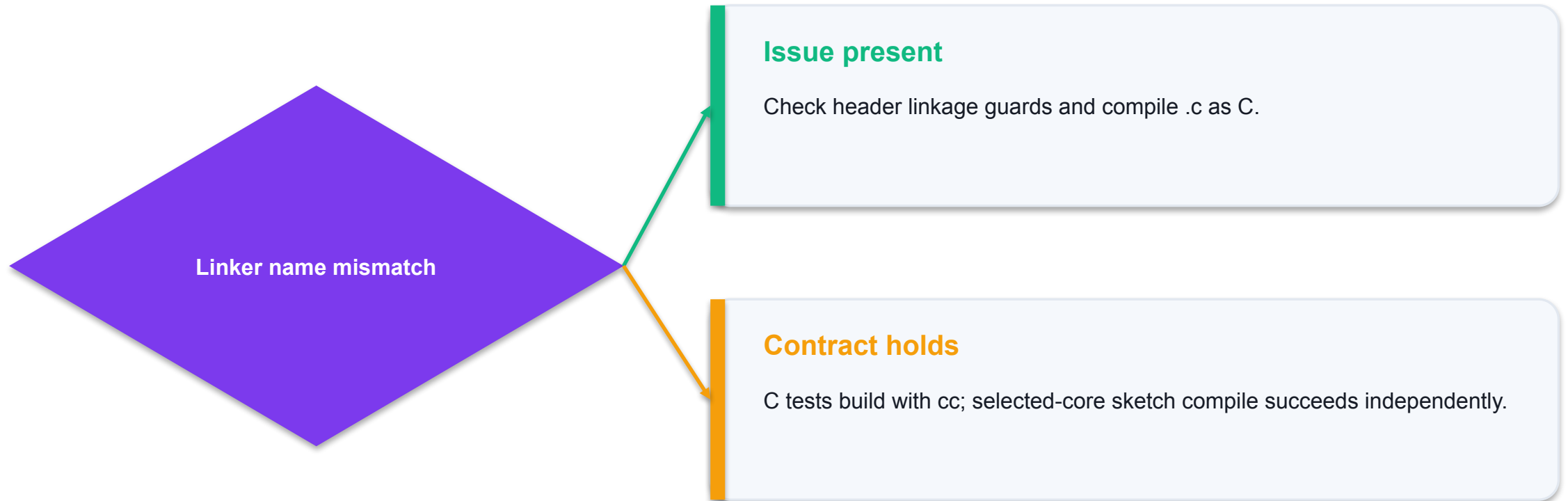
setup/loop and Arduino APIs stay in the sketch.

C logic lives behind a C++ sketch / contract

Contract field	Exact rule
logic.c	C11 host compilation; no Arduino includes.
logic.h	Shared declarations and fixed-width types.
sketch.ino	C++ runtime wrapper around pure logic.
Host test	Does not prove board upload or pin behaviour.

WHEN IT MATTERS C tests build with cc; selected-core sketch compile succeeds independently.

C logic lives behind a C++ sketch / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Variables encode value and unit

Exact mechanism / inspect the contract, not just syntax

```
#include <stdint.h>
int16_t temp_tenths = 250;
uint32_t sample_ms = 1000u;
uint8_t fan_request = 0u;
const int16_t on_tenths = 300;
```

Unit

250 means 25.0 degrees; encode the unit in the name.

Range

int16_t supports negatives and the chosen sensor range.

Time

Unsigned milliseconds support modular elapsed-time checks.

Constant

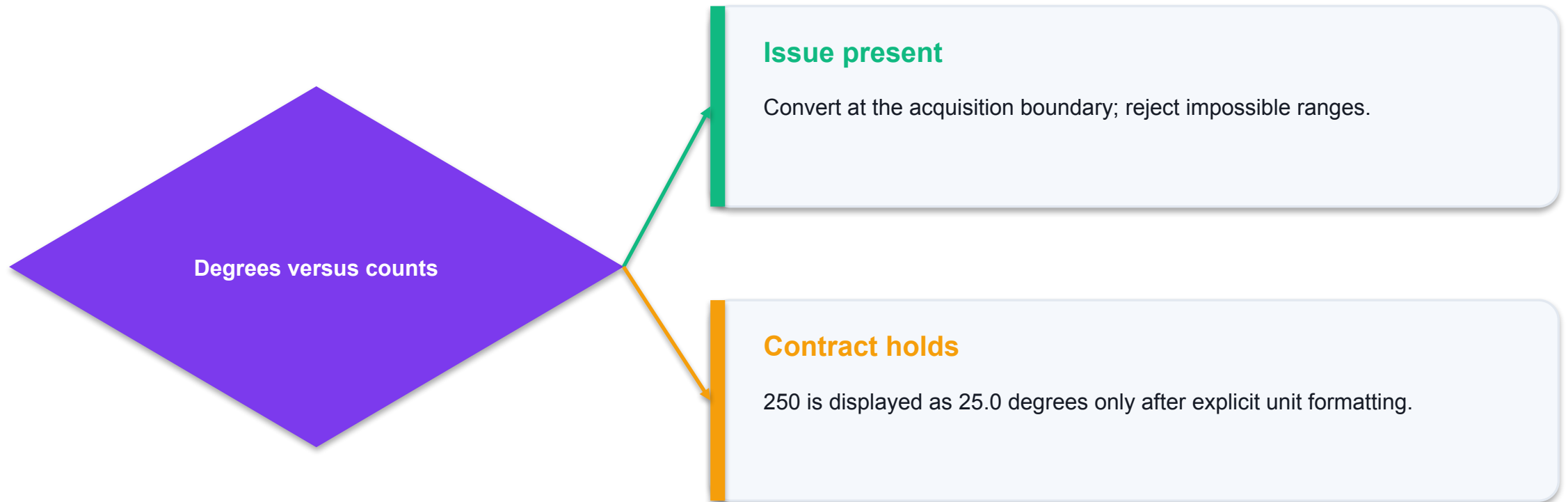
const describes intent, not necessarily a flash location.

Variables encode value and unit / contract

Contract field	Exact rule
Temperature	Allowed -400..1250 tenths in integration activity.
Timestamp	uint32_t wraps modulo 2^{32} .
Request	0/1 only; do not use a temperature as Boolean.
Platform	sizeof(int) is target-dependent; inspect instead of assuming.

WHEN IT MATTERS 250 is displayed as 25.0 degrees only after explicit unit formatting.

Variables encode value and unit / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Integer arithmetic needs promotion

Exact mechanism / inspect the contract, not just syntax

```
raw = 2048; adc_max = 4095;  
numerator = raw * 255 + adc_max/2;  
PWM = numerator / adc_max;  
PWM = 128;  
maximum numerator < 2^32
```

Promotion

Perform multiplication in uint32_t before narrowing.

Rounding

Adding half the denominator rounds to nearest.

Clamping

Negative and over-range raw values are bounded first.

Configuration

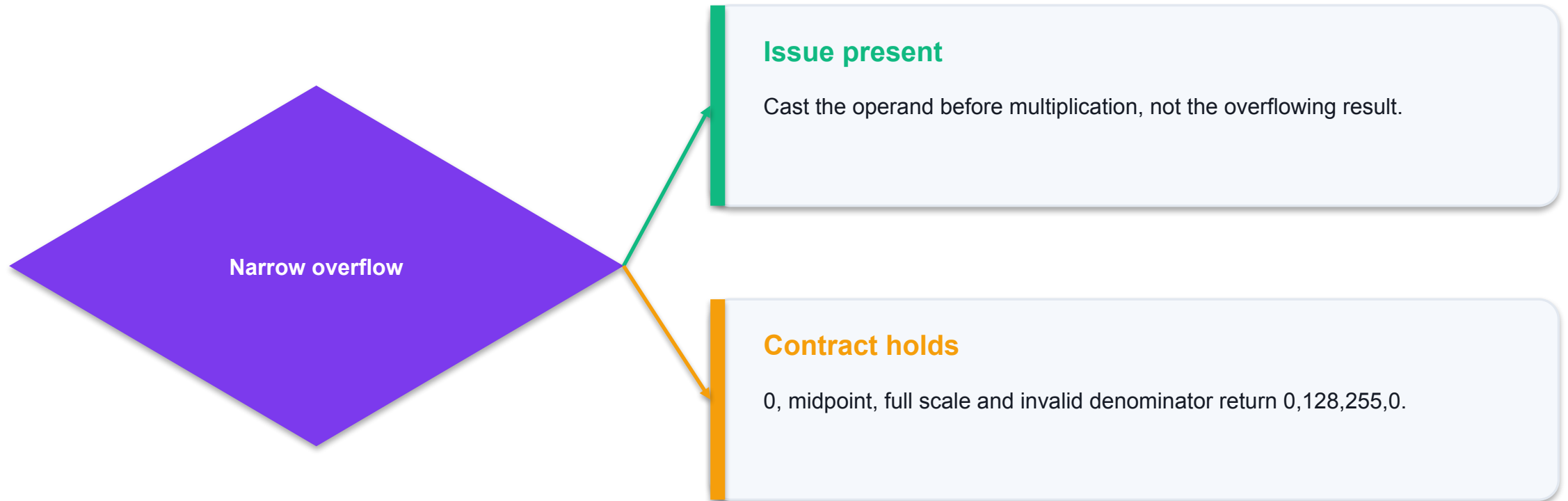
4095 is an explicit teaching setting, not an assumed default.

Integer arithmetic needs promotion / contract

Contract field	Exact rule
adc_max	1..65535; zero is invalid.
raw	Signed input clamped to 0..adc_max.
Result	uint8_t output 0..255.
Physical	PWM pin and frequency depend on selected core/pin.

WHEN IT MATTERS 0, midpoint, full scale and invalid denominator return 0,128,255,0.

Integer arithmetic needs promotion / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

A function makes state visible

Exact mechanism / inspect the contract, not just syntax

```
int alarm(int temperature, int limit) {  
    return temperature >= limit;  
}  
/* caller chooses the threshold */  
int request = alarm(30, 30);
```

Parameter

limit is passed, not read from a hidden global.

Return

Equality is defined as alarm-on.

Purity

No pin writes or logging inside the function.

Testability

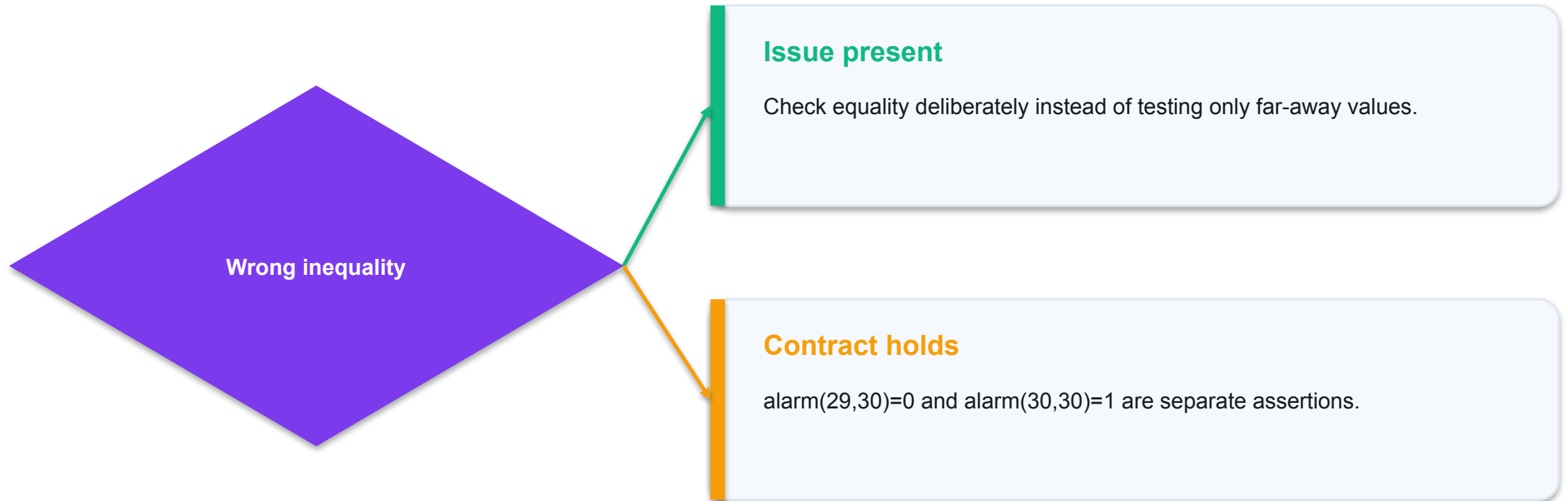
The same contract runs on a host and MCU.

A function makes state visible / contract

Contract field	Exact rule
Inputs	Signed whole degrees; both values share a unit.
Output	Exactly 0 or 1.
Side effects	None.
Boundary	29 below; 30 at; 31 above a limit of 30.

WHEN IT MATTERS `alarm(29,30)=0` and `alarm(30,30)=1` are separate assertions.

A function makes state visible / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Lifetime is not the same as scope

Exact mechanism / inspect the contract, not just syntax

```
static uint32_t last_ms;
void poll(uint32_t now) {
    int local_request = 0;
    if (due(now, &last_ms, 100u))
        local_request = 1;
    /* local_request dies on return */
}
```

Static

last_ms persists across calls and is zero-initialised.

Automatic

local_request exists only during the function call.

Scope

Both names are private to their declaration context.

Ownership

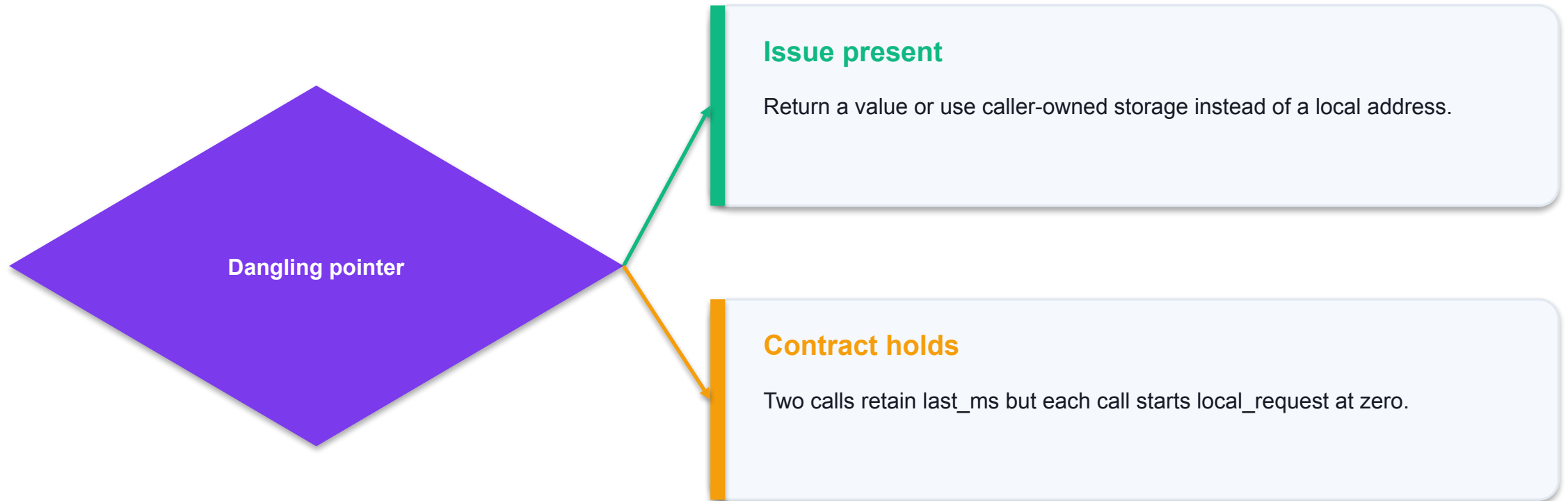
Persistent state must have a documented reset operation.

Lifetime is not the same as scope / contract

Contract field	Exact rule
Automatic storage	Never return the address of a local variable.
Static state	Survives calls; can complicate test isolation.
Explicit struct	Preferred when multiple instances are needed.
Initialisation	Tests establish a known initial state.

WHEN IT MATTERS Two calls retain `last_ms` but each call starts `local_request` at zero.

Lifetime is not the same as scope / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Compiler output is executable evidence

Exact mechanism / inspect the contract, not just syntax

```
source .c -> preprocessing  
translation -> object .o  
linking -> host executable  
warnings -> review gate  
assertions -> run-time evidence  
board compile -> separate target gate
```

Preprocess

Headers and macros become a translation unit.

Compile

Types and syntax are checked per unit.

Link

External references must resolve across units.

Run

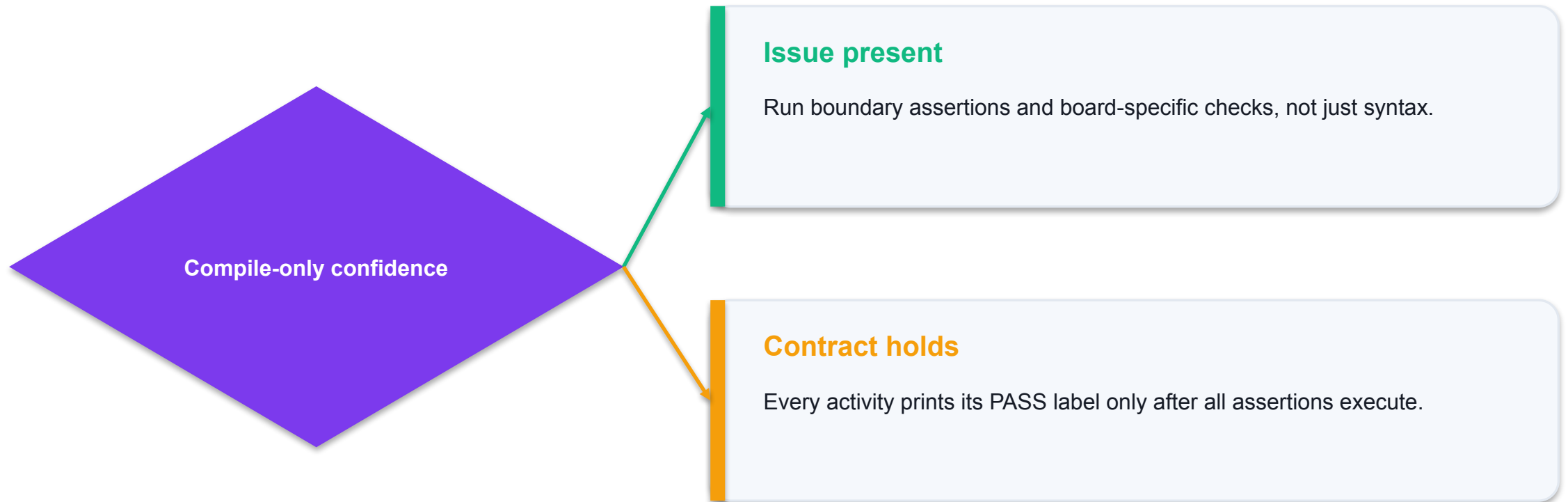
A compiled binary can still implement wrong requirements.

Compiler output is executable evidence / contract

Contract field	Exact rule
Host gate	-std=c11 -Wall -Wextra -Werror -pedantic.
Test gate	Nonzero exit means failed acceptance.
Board gate	Record board/core/library selection.
Evidence limit	Host compile is not measured hardware upload.

WHEN IT MATTERS Every activity prints its PASS label only after all assertions execute.

Compiler output is executable evidence / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Dynamic allocation has an explicit failure path

Exact mechanism / inspect the contract, not just syntax

```
int *values = malloc(count * sizeof *values);
if (values == NULL) { /* safe failure */ }
/* use only within validated count */
free(values);
values = NULL;
/* core activities prefer fixed arrays */
```

Allocation

Heap allocation can fail even after compilation succeeds.

Bounds

Validate count and product size before allocation.

Ownership

One owner releases the allocation exactly once.

Embedded

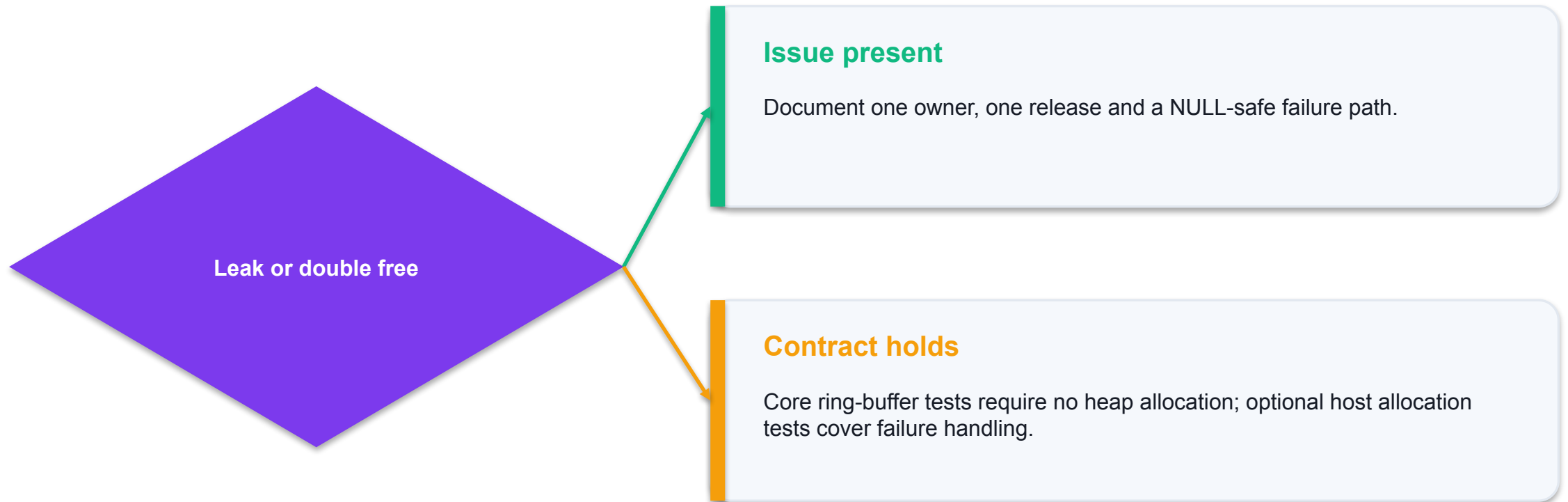
Fixed-capacity arrays avoid allocation in the control loop.

Dynamic allocation has an explicit failure path / contract

Contract field	Exact rule
Header	<stdlib.h> declares malloc/free.
Size	Check <code>count<=SIZE_MAX/sizeof *values</code> before multiplying.
Failure	NULL is not a valid array to index.
realloc	Use a temporary pointer; preserve original on failure.

WHEN IT MATTERS Core ring-buffer tests require no heap allocation; optional host allocation tests cover failure handling.

Dynamic allocation has an explicit failure path / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Specify and model a doorbell

ACTIVITY 1

Independent progressive practice /full procedures in guide

A visitor presses a button. The LED and a low-power buzzer request remain active for 2 seconds without freezing the loop.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Five assertions pass; activation is 1 before the deadline and 0 at it.

Specify and model a doorbell / runtime



EXACT CONTRACT

Input pressed is 0/1; timestamp is uint32_t milliseconds. A new press restarts the 2000 ms interval; output is 0 at the deadline.

Specify and model a doorbell / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int doorbell(uint32_t now, int pressed, uint32_t *started, int
*active) {
    if (pressed) { *started = now; *active = 1; }
    if (*active && (uint32_t)(now - *started) >= 2000u) *active =
0;
    return *active;
}
```

Requirements

Two outputs share one time contract.

State

started and active persist across calls.

Rollover

Unsigned subtraction measures elapsed time.

Acceptance

Check 1999 ms and 2000 ms separately.

Specify and model a doorbell / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Input pressed is 0/1; timestamp is uint32_t milliseconds. A new press restarts the 2000 ms interval; output is 0 at the deadline. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Specify and model a doorbell. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Specify and model a doorbell / acceptance

✓ Expected result

Five assertions pass; activation is 1 before the deadline and 0 at it. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Validate a GPIO truth table

ACTIVITY 2

Independent progressive practice /full procedures in guide

An active-low push button and an enable switch control a status indicator. Separate electrical level from logical meaning.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

All four truth-table combinations and one invalid value pass.

Validate a GPIO truth table / runtime



EXACT CONTRACT

raw_high 0 means pressed; enabled 0 always suppresses the output. Invalid levels return 0.

Validate a GPIO truth table / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int indicator(int raw_high, int enabled) {
    if ((raw_high != 0 && raw_high != 1) ||
        (enabled != 0 && enabled != 1)) return 0;
    return !raw_high && enabled;
}
```

Polarity

INPUT_PULLUP changes interpretation, not the Boolean requirement.

Truth table

Four valid input combinations exhaust the 2-bit domain.

Validation

Reject an impossible raw value.

Driver

Translate returned Boolean to digitalWrite in the adapter.

Validate a GPIO truth table / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: raw_high 0 means pressed; enabled 0 always suppresses the output. Invalid levels return 0. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Validate a GPIO truth table. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Validate a GPIO truth table / acceptance

✓ Expected result

All four truth-table combinations and one invalid value pass. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Schedule events without delay

ACTIVITY 3

Independent progressive practice /full procedures in guide

A status LED tick must not block sensor polling. Decide whether one periodic task is due from elapsed time.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Six checks pass including rollover and invalid period.

Schedule events without delay / runtime



EXACT CONTRACT

period must be nonzero and below 2^{31} ms; next execution follows the actual observed timestamp (no backlog replay).

Schedule events without delay / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int due(uint32_t now, uint32_t *last, uint32_t period) {
    if (period == 0u || period >= 0x80000000u) return 0;
    if ((uint32_t)(now - *last) < period) return 0;
    *last = now; return 1;
}
```

Time

now-last is modular arithmetic.

Policy

Reset to now prevents bursts after a late loop.

Latency

Period is not an exact loop execution guarantee.

Invariant

No call may block waiting for the period.

Schedule events without delay / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: period must be nonzero and below 2^{31} ms; next execution follows the actual observed timestamp (no backlog replay). Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Schedule events without delay. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Schedule events without delay / acceptance

✓ Expected result

Six checks pass including rollover and invalid period. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

TOPIC 2 /K2/A2

Modular development and AI-assisted review

Activities 4-6 / specifications, contracts, failure and evidence

Header guards share one declaration

Exact mechanism / inspect the contract, not just syntax

```
#ifndef SENSOR_LOGIC_H
#define SENSOR_LOGIC_H
#ifdef __cplusplus
extern "C" {
#endif
int alarm(int temperature,int limit);
/* close extern C and guard */
```

Guard

Prevents duplicate declarations of types in one translation unit.

Declaration

Header specifies the callable interface.

Implementation

Function body belongs in logic.c.

Linkage

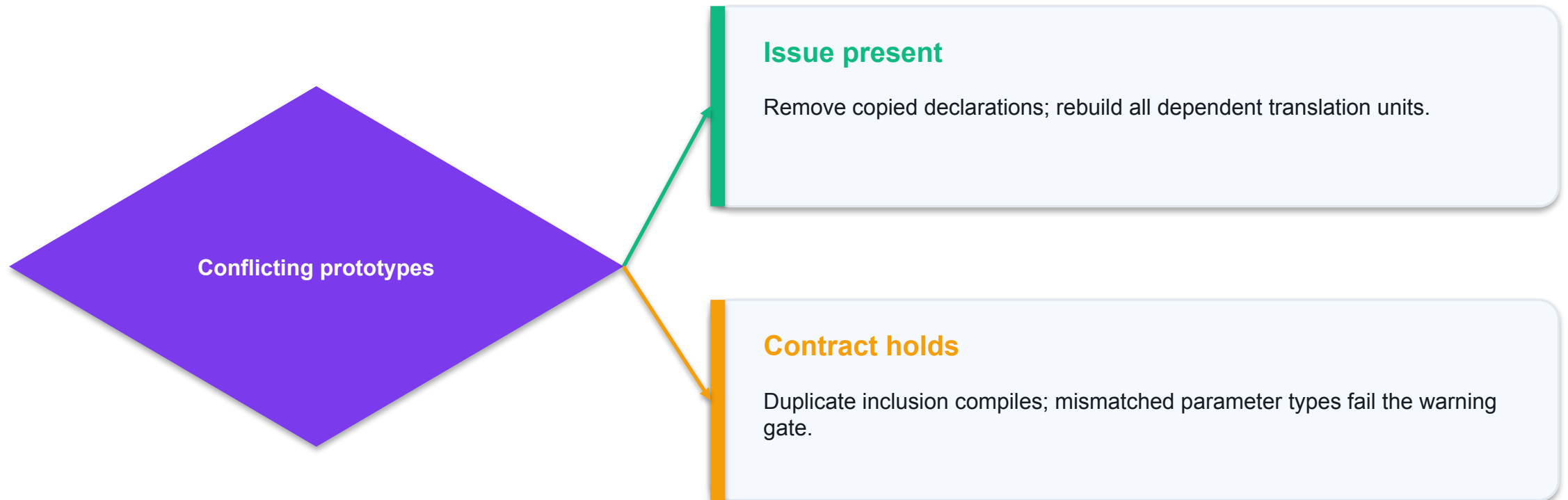
A C++ caller sees C linkage only inside the guard.

Header guards share one declaration / contract

Contract field	Exact rule
Include	Both implementation and caller include the same header.
Types	Use matching types in declaration and definition.
Constants	Avoid multiple mutable global definitions in a header.
Build proof	Include the header twice in the host test.

WHEN IT MATTERS Duplicate inclusion compiles; mismatched parameter types fail the warning gate.

Header guards share one declaration / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

AI review is bounded by evidence

Exact mechanism / inspect the contract, not just syntax

```
Input: specification + logic.h
Ask: find overflow and edge cases
Output: patch + stated assumptions
Human: compare interface and units
Compiler: strict warnings
Tests: boundaries and invalid inputs
```

Context

Supply exact contracts and target constraints, not just a task name.

Review

Require explanations tied to lines and requirements.

Privacy

Use mock sensor data; omit credentials and client identities.

Authority

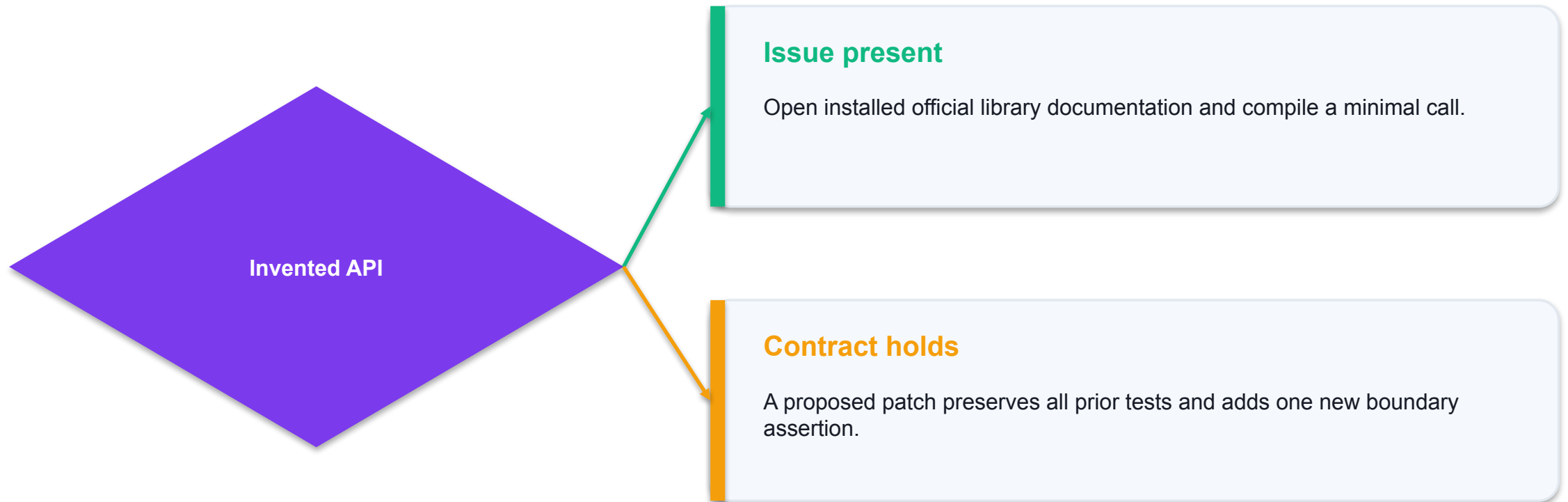
The assistant proposes; maintainer accepts only verified changes.

AI review is bounded by evidence / contract

Contract field	Exact rule
Permitted	Explain, refactor and propose tests.
Forbidden	Silently change thresholds, units or hardware pins.
Evidence	Save prompt, proposal, decision and test outcome.
Failure mode	Plausible generated code can still be unsafe.

WHEN IT MATTERS A proposed patch preserves all prior tests and adds one new boundary assertion.

AI review is bounded by evidence / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Separate acquisition from conversion

Exact mechanism / inspect the contract, not just syntax

```
analogRead(A0) -> raw count  
light_percent(raw, maximum)  
validated percentage -> Serial  
host mock -> same conversion  
MCU wrapper -> same C logic
```

Acquisition

Arduino API reads a hardware-dependent count.

Conversion

C logic uses an explicit maximum.

Presentation

Serial output is outside the pure function.

Calibration

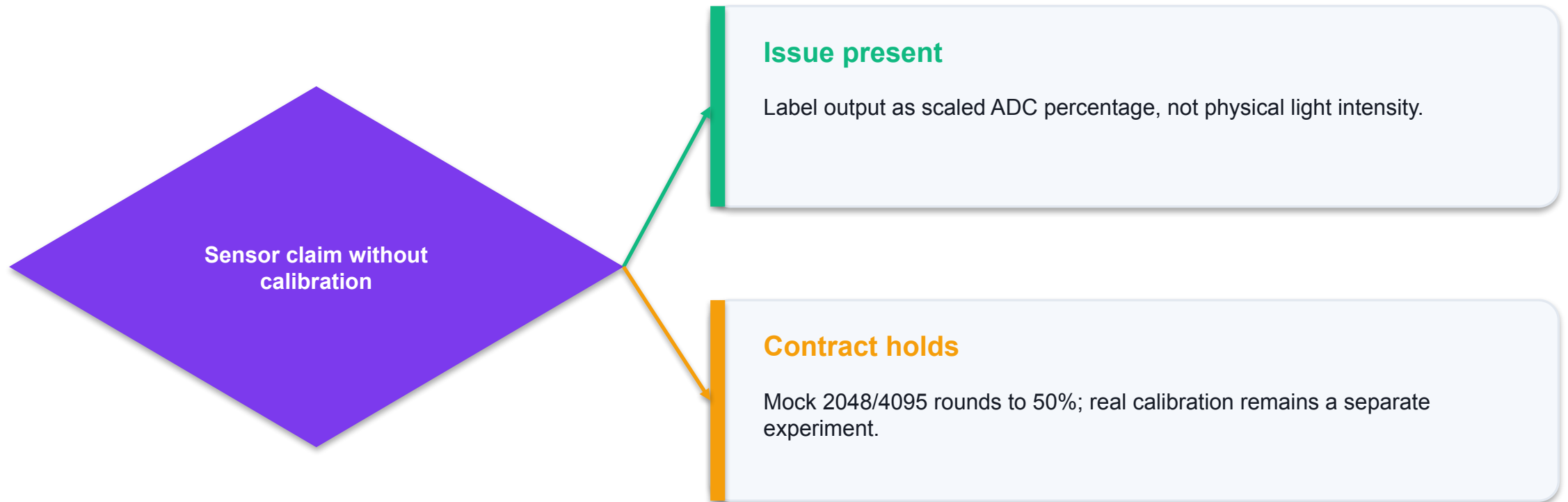
A percentage of ADC range is not lux.

Separate acquisition from conversion / contract

Contract field	Exact rule
Raw value	Count in a configured resolution range.
maximum	Explicit denominator, 1..65535.
Return	-1 for invalid maximum; otherwise 0..100.
Documentation	Record units at each boundary.

WHEN IT MATTERS Mock 2048/4095 rounds to 50%; real calibration remains a separate experiment.

Separate acquisition from conversion / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Macros can evaluate twice

Exact mechanism / inspect the contract, not just syntax

```
#define BAD_MAX(a,b) ((a)>(b)?(a):(b))  
/* BAD_MAX(i++, 10) is unsafe */  
static inline int max_int(int a,int b) {  
    return a>b ? a:b;  
}
```

Expansion

A macro substitutes text before compilation.

Side effect

`i++` can appear in condition and selected branch.

Inline

Function arguments are evaluated once each.

Parentheses

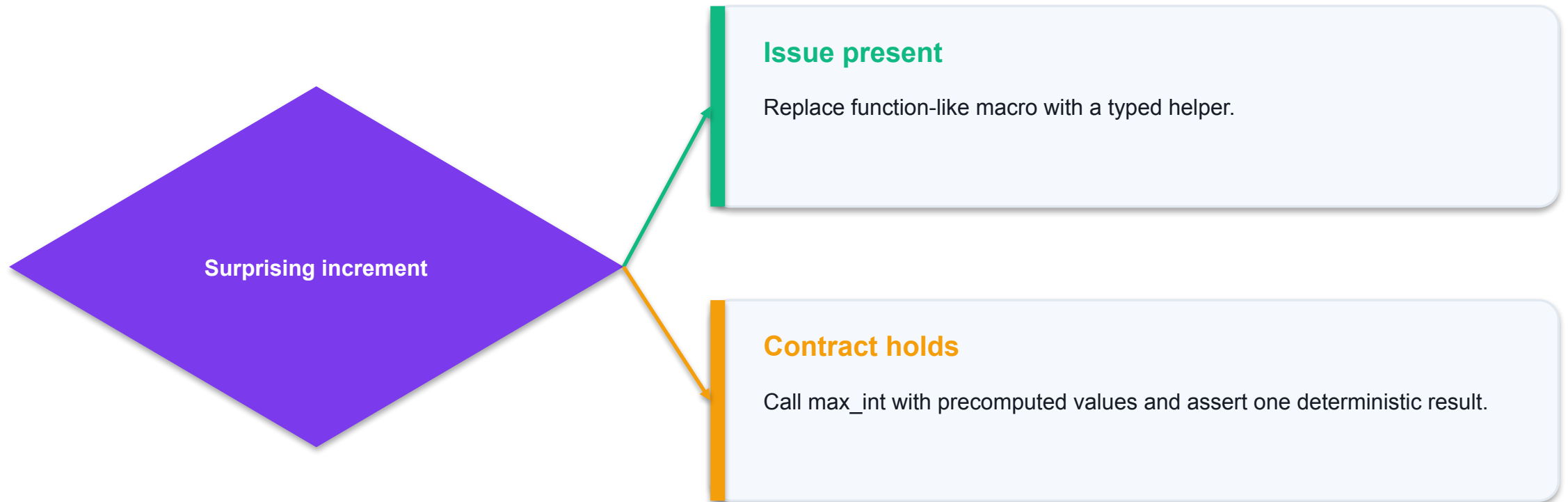
Protect precedence but do not fix repeated evaluation.

Macros can evaluate twice / contract

Contract field	Exact rule
Macro argument	Never pass an increment to a repeated-argument macro.
Constant macro	Suitable for a compile-time numeric setting.
Typed function	Enforces parameter and return types.
Review	Inspect preprocessed meaning, not only the macro name.

WHEN IT MATTERS Call `max_int` with precomputed values and assert one deterministic result.

Macros can evaluate twice / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Preprocessor choices change the build

Exact mechanism / inspect the contract, not just syntax

```
#ifndef SAMPLE_PERIOD_MS
#define SAMPLE_PERIOD_MS 100u
#endif
#if SAMPLE_PERIOD_MS == 0
#error Sample period must be nonzero
#endif
```

Configuration

A build flag can override the fallback constant.

Validation

Reject invalid configuration before runtime.

Conditional

#if selects compiled text, not a runtime branch.

Traceability

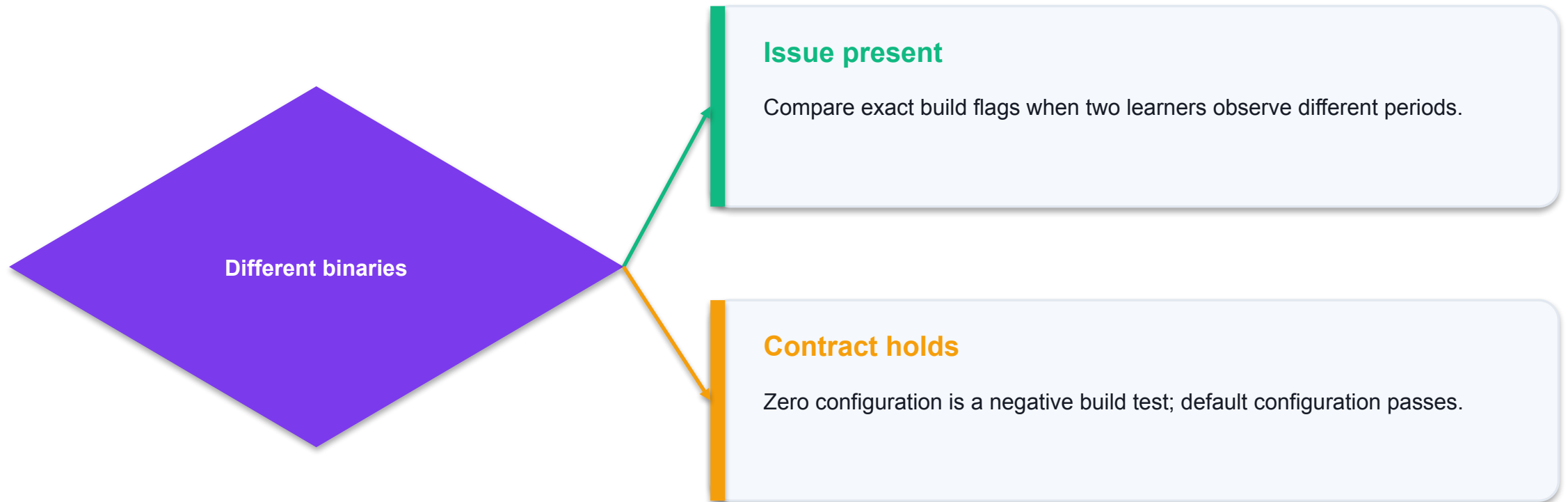
Record flags in the build evidence.

Preprocessor choices change the build / contract

Contract field	Exact rule
Default	100 milliseconds is the teaching period.
Override	Compiler -D is a build configuration.
Invalid	Zero must fail preprocessing.
Portability	No board identity is inferred from a macro alone.

WHEN IT MATTERS Zero configuration is a negative build test; default configuration passes.

Preprocessor choices change the build / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Warnings expose narrowing mistakes

Exact mechanism / inspect the contract, not just syntax

```
uint32_t raw = 65535u;  
uint32_t scaled = raw * 255u;  
uint8_t request = (uint8_t)255u;  
/* not: uint16_t scaled = raw*255 */  
/* validate before explicit narrowing */
```

Intermediate

scaled needs more than 16 bits at full raw scale.

Conversion

A cast can hide warnings without proving correctness.

Validation

Narrow only after proving range 0..255.

Target

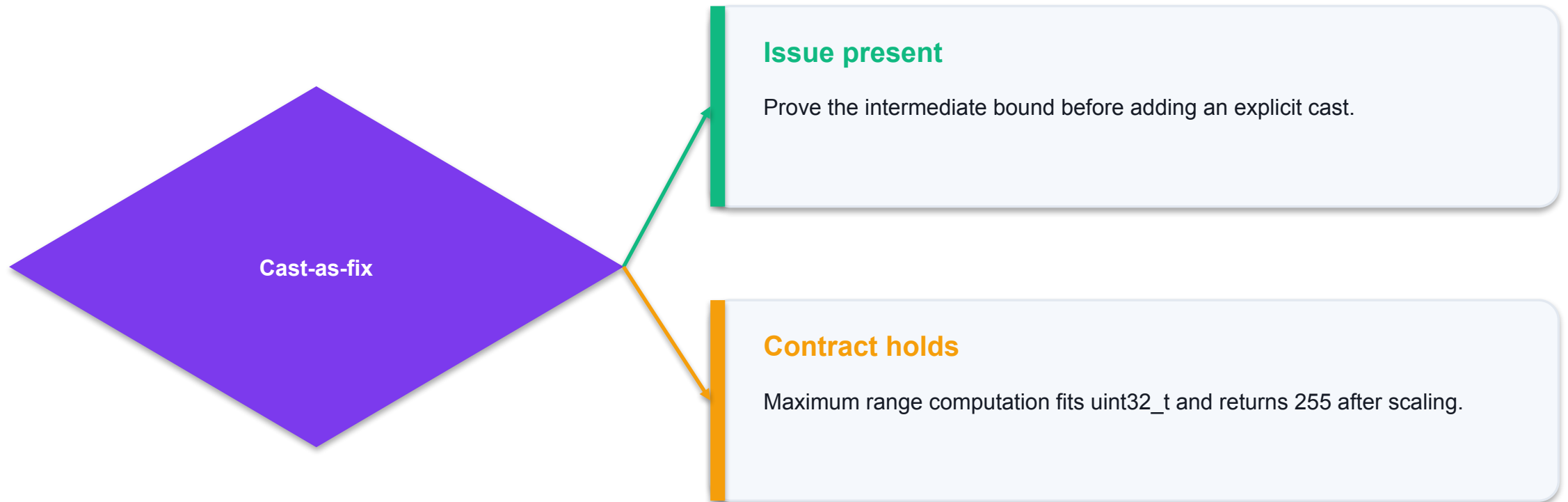
Integer sizes vary; fixed-width intermediates express intent.

Warnings expose narrowing mistakes / contract

Contract field	Exact rule
Maximum product	$65535 * 255 = 16711425$.
Result range	0..255 after division.
Warning gate	Enable strict baseline and review extra conversion warnings.
Assertion	Test maximum input, not only small values.

WHEN IT MATTERS Maximum range computation fits `uint32_t` and returns 255 after scaling.

Warnings expose narrowing mistakes / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Documentation comments state contracts

Exact mechanism / inspect the contract, not just syntax

```
/* percent: 0..100; age: milliseconds  
 * returns 1 only when dry and fresh  
 * invalid inputs return 0  
 * no GPIO side effects  
 */  
int water_request(int percent,unsigned age,int ok);
```

User need

Dry soil can request watering; faults must suppress it.

Unit

Percentage and milliseconds are explicit.

Return

0 is safe-off, including invalid data.

Limit

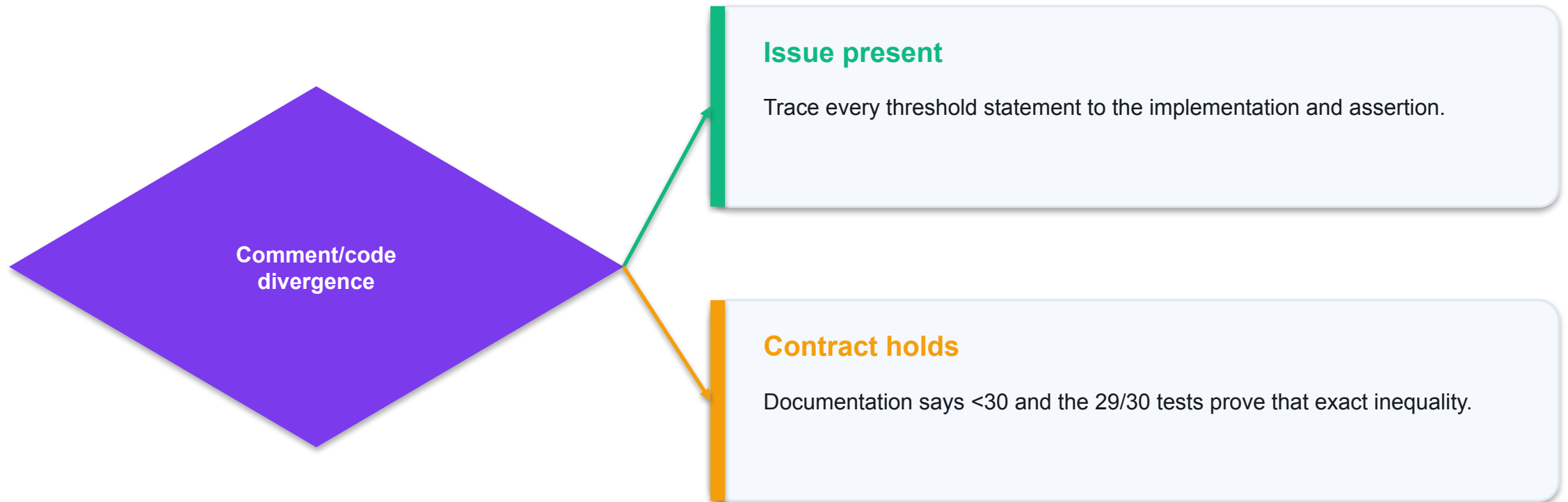
A request function is not a complete pump safety controller.

Documentation comments state contracts / contract

Contract field	Exact rule
Header comment	Defines public preconditions and output.
Implementation comment	Explains non-obvious invariants, not syntax.
Report	States limits such as missing maximum runtime.
Change	Update tests and docs when threshold changes.

WHEN IT MATTERS Documentation says <30 and the 29/30 tests prove that exact inequality.

Documentation comments state contracts / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Host files are not MCU storage APIs

Exact mechanism / inspect the contract, not just syntax

```
void read_mock(void) {  
    FILE *fp = fopen("samples.csv","r");  
    if (fp == NULL) return; /* stop: no valid handle */  
    char line[64];  
    while (fgets(line,sizeof line,fp)) {  
        /* validate complete bounded record */  
    }  
    fclose(fp);  
}
```

Host

stdio file I/O reads local mock CSV in a desktop program.

Bound

fgets limits bytes; a truncated line still needs rejection.

Error

Check open/read/close outcomes as required.

MCU

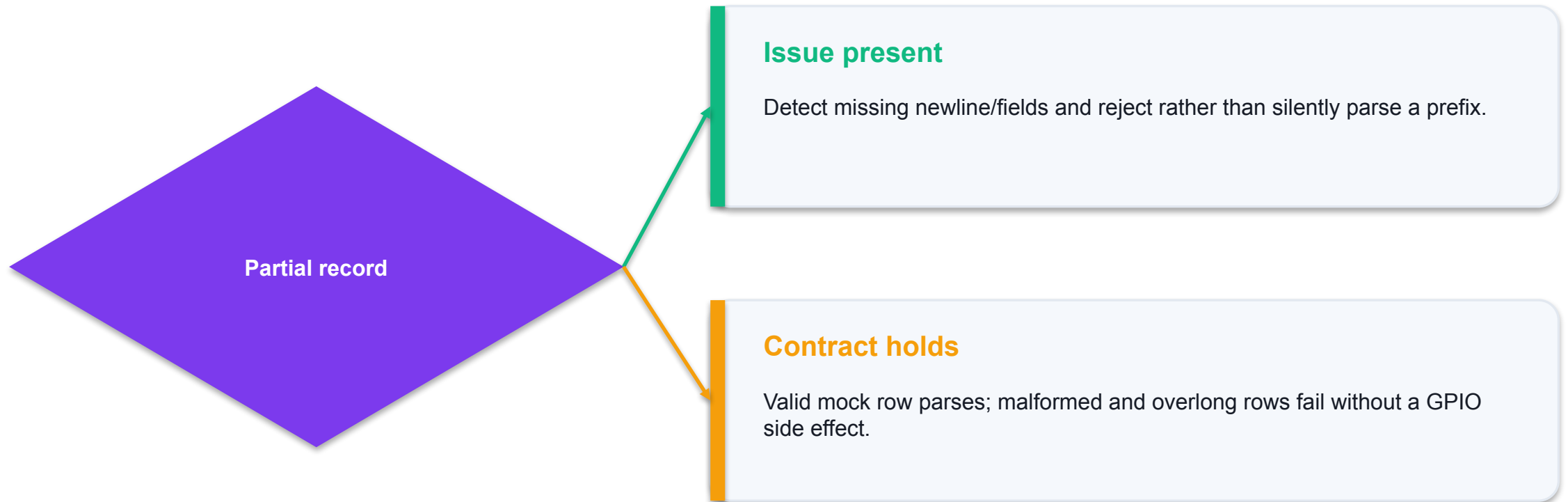
SD logging uses a board/library-specific adapter, not assumed fopen.

Host files are not MCU storage APIs / contract

Contract field	Exact rule
Input	Local mock data; no real personal records.
EOF	End-of-file differs from malformed line.
Parser	Check conversion count and permitted range.
Target	Physical SD write/readback is separate evidence.

WHEN IT MATTERS Valid mock row parses; malformed and overlong rows fail without a GPIO side effect.

Host files are not MCU storage APIs / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Version evidence identifies the build

Exact mechanism / inspect the contract, not just syntax

```
course version: 2.0
source revision: recorded at release
board model: exact variant
core and library versions: recorded
host compiler: recorded
hardware result: NOT RUN until measured
```

Artifact

A version label identifies course content, not target firmware SHA.

Source

Record the reviewed revision when available.

Environment

Core/library differences can change supported APIs.

Honesty

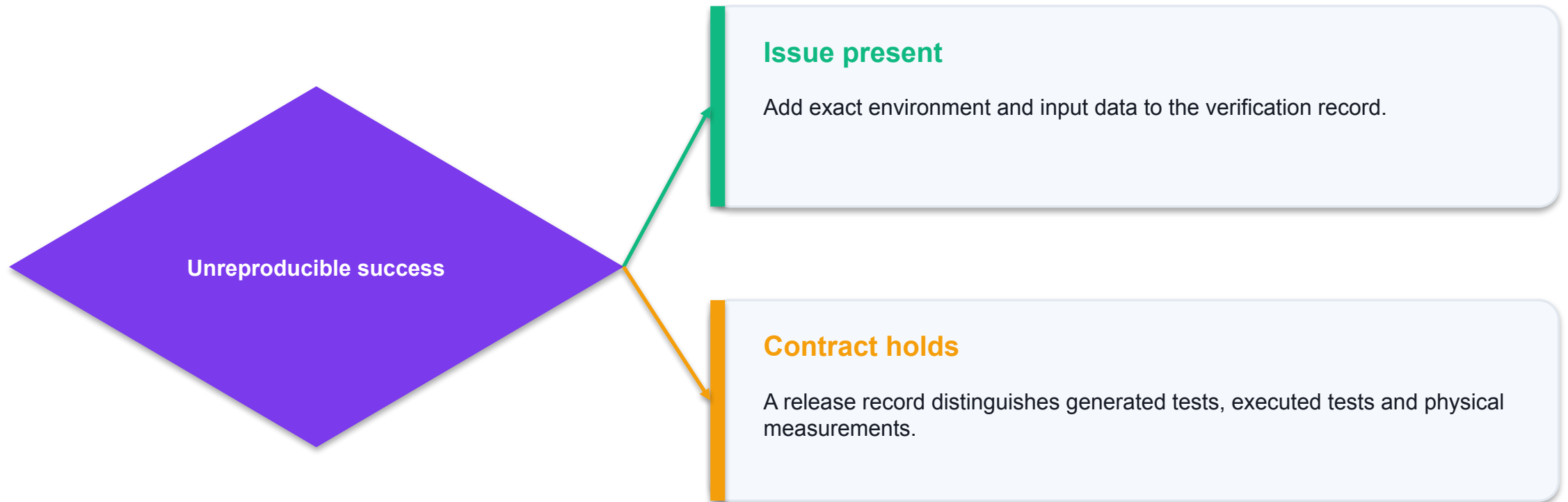
NOT RUN is evidence of a limit, not a failure to conceal.

Version evidence identifies the build / contract

Contract field	Exact rule
Host record	Compiler version plus test command/output.
MCU record	Board/core and sketch compile result.
Physical record	Wiring/photo/readback/timestamps.
Reproduction	Another learner can identify the same configuration.

WHEN IT MATTERS A release record distinguishes generated tests, executed tests and physical measurements.

Version evidence identifies the build / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

C++ references wrap caller-owned state

Exact mechanism / inspect the contract, not just syntax

```
class SensorView {  
    const Sample& sample;  
public:  
    explicit SensorView(const Sample& s):sample(s) {}  
    int temperature() const { return sample.temp_tenths; }  
};
```

Reference

A reference names an existing object; it is not an owning copy.

Lifetime

Caller-owned Sample must outlive SensorView.

Const

View cannot modify the referenced sample.

Scope

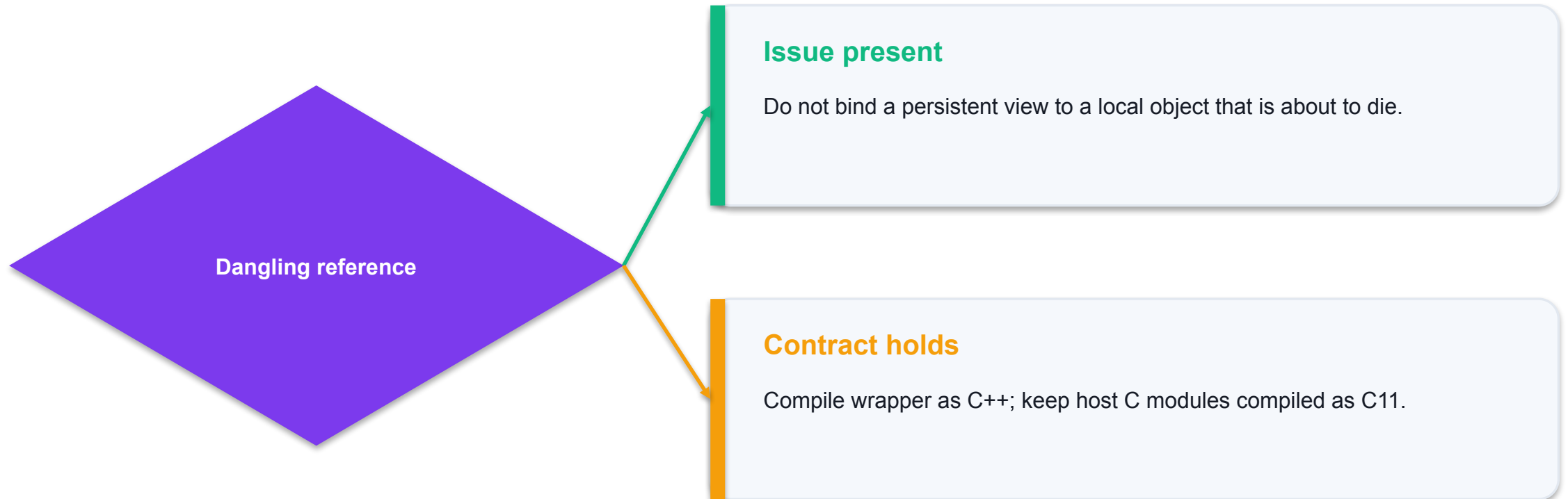
This lightweight C++ adapter does not replace the C core.

C++ references wrap caller-owned state / contract

Contract field	Exact rule
C core	Sample and functions remain C-compatible.
C++ wrapper	Classes/references belong in .cpp or .ino, not logic.c.
Ownership	No allocation is required for this view.
Getter	Returns a value and documents temperature unit.

WHEN IT MATTERS Compile wrapper as C++; keep host C modules compiled as C11.

C++ references wrap caller-owned state / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

RAII ties a resource to object lifetime

Exact mechanism / inspect the contract, not just syntax

```
class HostFile {  
    FILE* fp;  
public:  
    explicit HostFile(const char* path):fp(fopen(path,"r")) {}  
    ~HostFile() { if(fp) fclose(fp); }  
    HostFile(const HostFile&) = delete;  
    HostFile& operator=(const HostFile&) = delete;  
};
```

Acquire

Constructor tries to open a host file.

Release

Destructor closes an acquired resource.

Copy

Disabled copying prevents two owners closing one handle.

Boundary

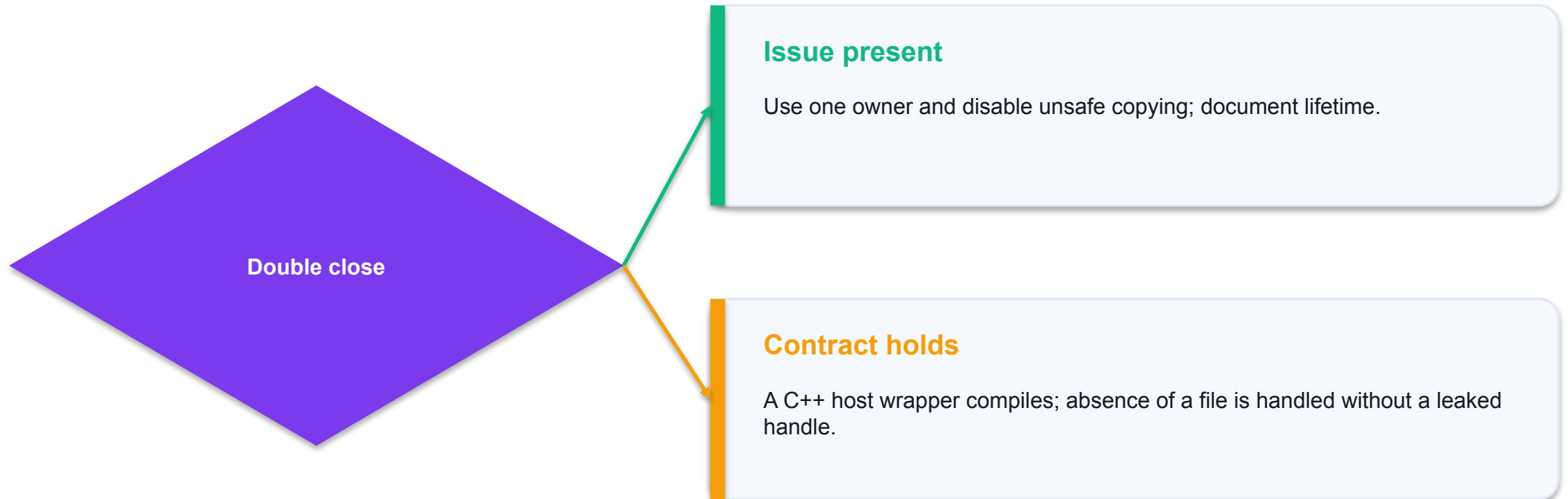
Host example teaches RAII, not an assumed MCU SD API.

RAII ties a resource to object lifetime / contract

Contract field	Exact rule
Language	RAII/class/destructor are C++ concepts.
Failure	Constructor must expose/check unsuccessful open.
Ownership	One object owns the handle.
Target	Use actual selected SD library in board adapters.

WHEN IT MATTERS A C++ host wrapper compiles; absence of a file is handled without a leaked handle.

RAII ties a resource to object lifetime / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

C and C++ compile different source contracts

Exact mechanism / inspect the contract, not just syntax

```
logic.c: cc -std=c11
adapter.cpp: c++ -std=c++17
logic.h: guarded extern "C"
.ino: Arduino-generated C++ translation
no classes/references/destructors in C module
```

C

Structures, pointers and functions form the hardware-independent core.

C++

References, const member functions and RAII fit thin adapters.

Linkage

extern C avoids a C++ symbol-name mismatch.

Education

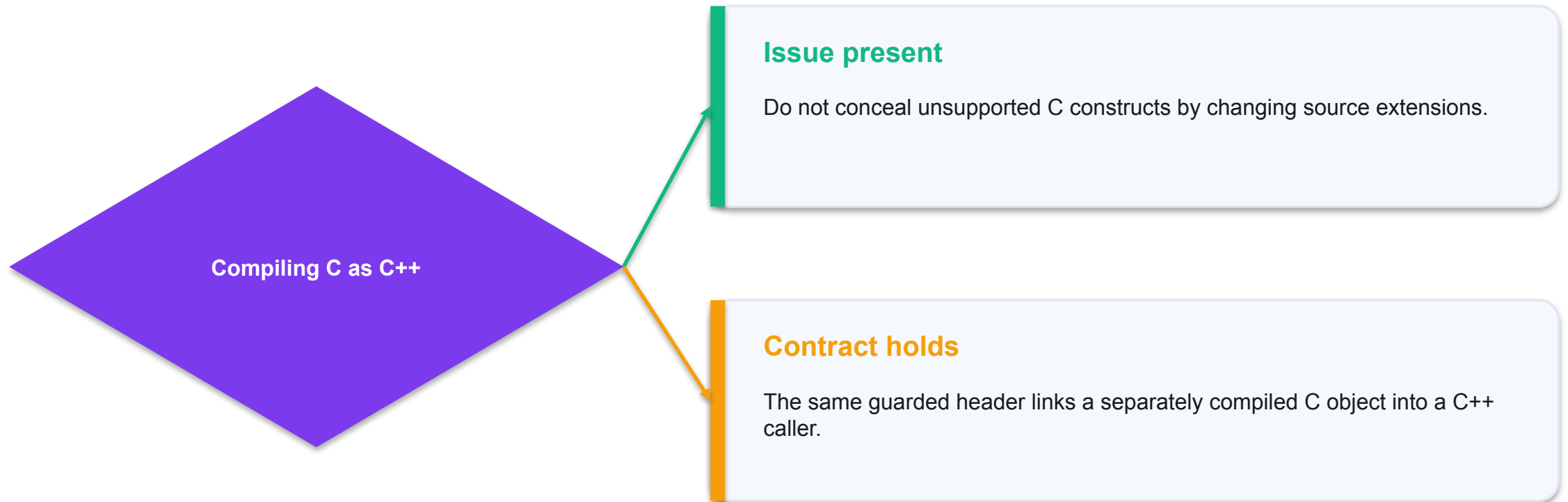
Google C++ material and requested tutorials supplement, not replace assessed C.

C and C++ compile different source contracts / contract

Contract field	Exact rule
Google primary	developers.google.com/edu/c++/getting-started .
Requested context	learn-cpp.org and learncpp.com learning references.
Core scope	All five approved software-design outcomes retained.
Evidence	Compile C and C++ units with the correct tools.

WHEN IT MATTERS The same guarded header links a separately compiled C object into a C++ caller.

C and C++ compile different source contracts / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Scale an ADC reading to PWM

ACTIVITY 4

Independent progressive practice /full procedures in guide

A greenhouse light monitor maps a configurable ADC range to a bounded PWM command without assuming UNO R3 ADC resolution.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Seven assertions pass; midpoint 2048/4095 maps to 128.

Scale an ADC reading to PWM / runtime



EXACT CONTRACT

raw is clamped to 0..adc_max; adc_max 1..65535; result rounded to nearest 0..255. Return 0 for invalid adc_max.

Scale an ADC reading to PWM / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
uint8_t pwm_scale(int32_t raw, uint32_t adc_max) {
    if (adc_max == 0 || adc_max > 65535u) return 0;
    if (raw < 0) raw = 0;
    if ((uint32_t)raw > adc_max) raw = (int32_t)adc_max;
    return (uint8_t)(((uint32_t)raw * 255u + adc_max/2u) /
adc_max);
}
```

Configuration

adc_max belongs to the selected resolution contract.

Promotion

Multiply in uint32_t, not a narrow integer.

Rounding

Half denominator implements nearest integer.

Bounds

Clamp before conversion to uint8_t.

Scale an ADC reading to PWM / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: raw is clamped to 0..adc_max; adc_max 1..65535; result rounded to nearest 0..255. Return 0 for invalid adc_max. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Scale an ADC reading to PWM. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Scale an ADC reading to PWM / acceptance

✓ Expected result

Seven assertions pass; midpoint 2048/4095 maps to 128. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Review modular sensor conversion

ACTIVITY 5

Independent progressive practice /full procedures in guide

An AI assistant proposes a light percentage conversion. Keep raw acquisition in the Arduino wrapper and pure conversion in a C module.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Five assertions pass; invalid denominator returns -1.

Review modular sensor conversion / runtime



EXACT CONTRACT

Maximum 1..65535; clamped raw input; rounded integer percentage 0..100. No Serial, GPIO or allocation in this module.

Review modular sensor conversion / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int light_percent(int raw, unsigned maximum) {
    if (maximum==0 || maximum>65535u) return -1;
    if (raw<0) raw=0;
    if ((unsigned)raw>maximum) raw=(int)maximum;
    return (int)(((unsigned)raw*100u+maximum/2u)/maximum);
}
```

Separation

Arduino acquisition supplies an integer.

Interface

Header declares the conversion contract.

Review

AI suggestions must preserve invalid-input semantics.

Proof

Boundary tests establish scaling, not sensor calibration.

Review modular sensor conversion / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Maximum 1..65535; clamped raw input; rounded integer percentage 0..100. No Serial, GPIO or allocation in this module. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Review modular sensor conversion. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Review modular sensor conversion / acceptance

✓ Expected result

Five assertions pass; invalid denominator returns -1. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Repair a header and build contract

ACTIVITY 6

Independent progressive practice /full procedures in guide

Two developers duplicate a threshold constant and introduce an incompatible function declaration. Restore one interface and guarded header.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Four assertions and duplicate-header inclusion compile successfully.

Repair a header and build contract / runtime



EXACT CONTRACT

alarm() accepts signed whole-degree temperature and limit; equality is alarm-on. No hidden global threshold.

Repair a header and build contract / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int alarm(int temperature, int limit) { return temperature >=
limit; }
```

Header

A declaration is shared, not copied implementation.

Guard

Multiple includes must not redefine types.

Linkage

extern "C" guards let a C++ .ino call compiled .c.

Warnings

Treat conflicting declarations as a failed build.

Repair a header and build contract / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: alarm() accepts signed whole-degree temperature and limit; equality is alarm-on. No hidden global threshold. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Repair a header and build contract. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Repair a header and build contract / acceptance

✓ Expected result

Four assertions and duplicate-header inclusion compile successfully. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

DAY1 RECAP /CODE

One contract survives two targets.

Host C tests establish logic; the selected-core C++ wrapper establishes target integration. Physical behaviour still needs a board.

TOPIC 3 /K3/A3

Controls, memory and data structures

Activities 7-9 / specifications, contracts, failure and evidence

If-else encodes a boundary policy

Exact mechanism / inspect the contract, not just syntax

```
if (temp_tenths >= 300) return 1;  
if (temp_tenths <= 280) return 0;  
return previous != 0;
```

Upper boundary

At 30.0 degrees the fan request turns on.

Lower boundary

At 28.0 degrees the request turns off.

Deadband

Between thresholds the prior state matters.

Sequence

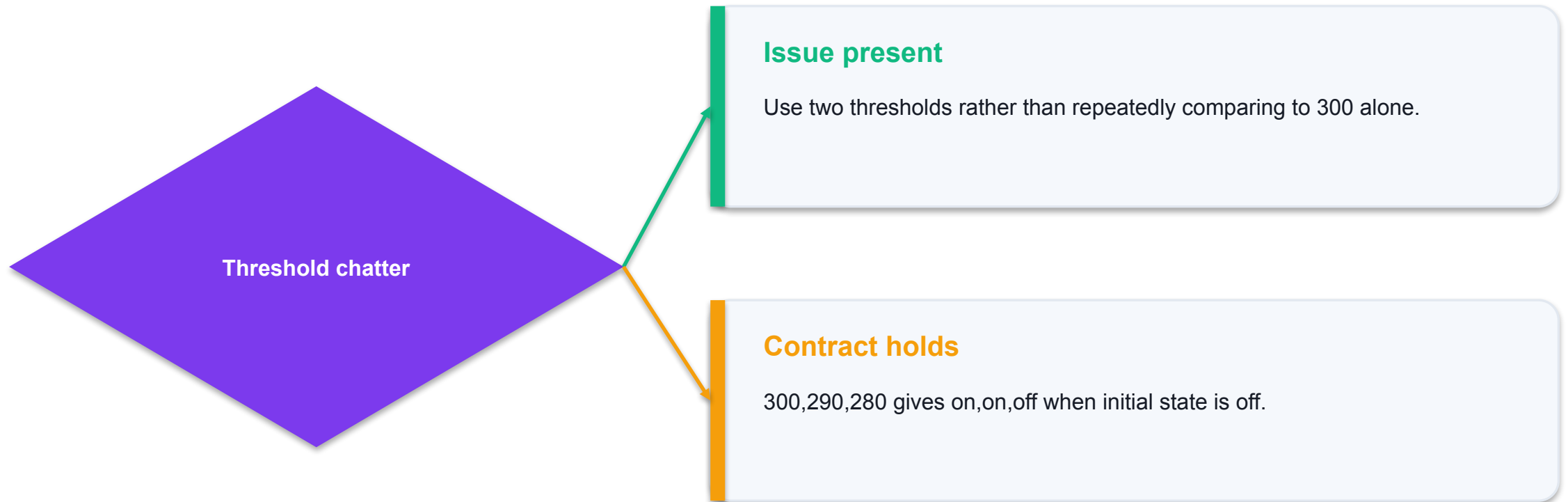
First-match order must implement the chosen policy.

If-else encodes a boundary policy / contract

Contract field	Exact rule
On threshold	≥ 300 tenths.
Off threshold	≤ 280 tenths.
Middle	281..299 retain previous state.
Reset	Initial prior state must be chosen explicitly.

WHEN IT MATTERS 300,290,280 gives on,on,off when initial state is off.

If-else encodes a boundary policy / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Switch-case selects operating modes

Exact mechanism / inspect the contract, not just syntax

```
switch (mode) {  
  case 0: return automatic_request;  
  case 1: return 1;  
  case 2: return 0;  
  default: return 0;  
}
```

Mode

AUTO, MANUAL_ON and OFF have distinct intent.

Default

Unknown values fail off instead of inheriting a stale mode.

Return

Each branch ends without accidental fallthrough.

Validation

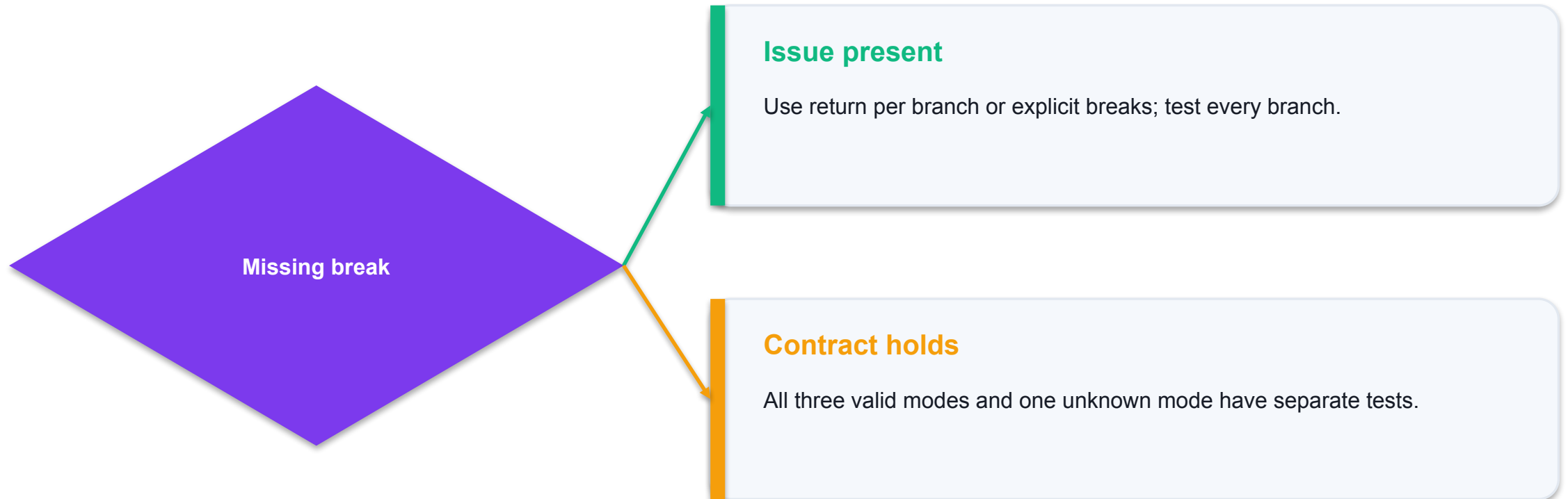
A physical toggle encoding must map to these values explicitly.

Switch-case selects operating modes / contract

Contract field	Exact rule
0	Use threshold/hysteresis logic.
1	Manual request on; hardware safety gate still applies.
2	Request off.
Other	Request off and record invalid mode.

WHEN IT MATTERS All three valid modes and one unknown mode have separate tests.

Switch-case selects operating modes / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

For loops walk bounded arrays

Exact mechanism / inspect the contract, not just syntax

```
int sum=0;
for (unsigned i=0;i<count;i++) {
    sum += values[i];
}
/* count must be <= capacity */
```

Initialisation

i starts at first valid element.

Condition

i<count excludes the one-past-end index.

Progress

Increment occurs once per iteration.

Invariant

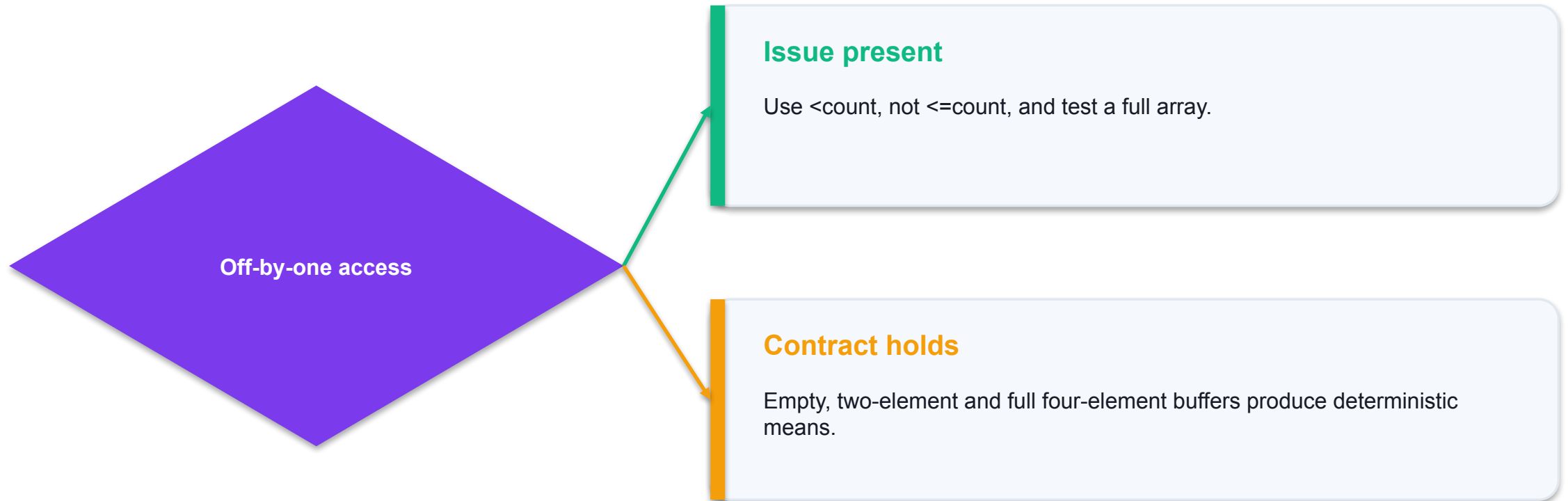
count cannot exceed the allocated capacity.

For loops walk bounded arrays / contract

Contract field	Exact rule
Array capacity	4 int elements in the ring buffer.
Valid count	0..4.
Empty	Mean returns 0 before division.
Sum bound	Four values each -1000..1000 yield -4000..4000.

WHEN IT MATTERS Empty, two-element and full four-element buffers produce deterministic means.

For loops walk bounded arrays / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

While and do-while differ at entry

Exact mechanism / inspect the contract, not just syntax

```
while (available > 0) {  
  /* consume one bounded item */  
  available--;  
}  
do { /* one trial */ } while (retry);
```

while

May execute zero times when no data exists.

do-while

Always executes once before checking.

Progress

Each iteration must consume or change its termination state.

Embedded

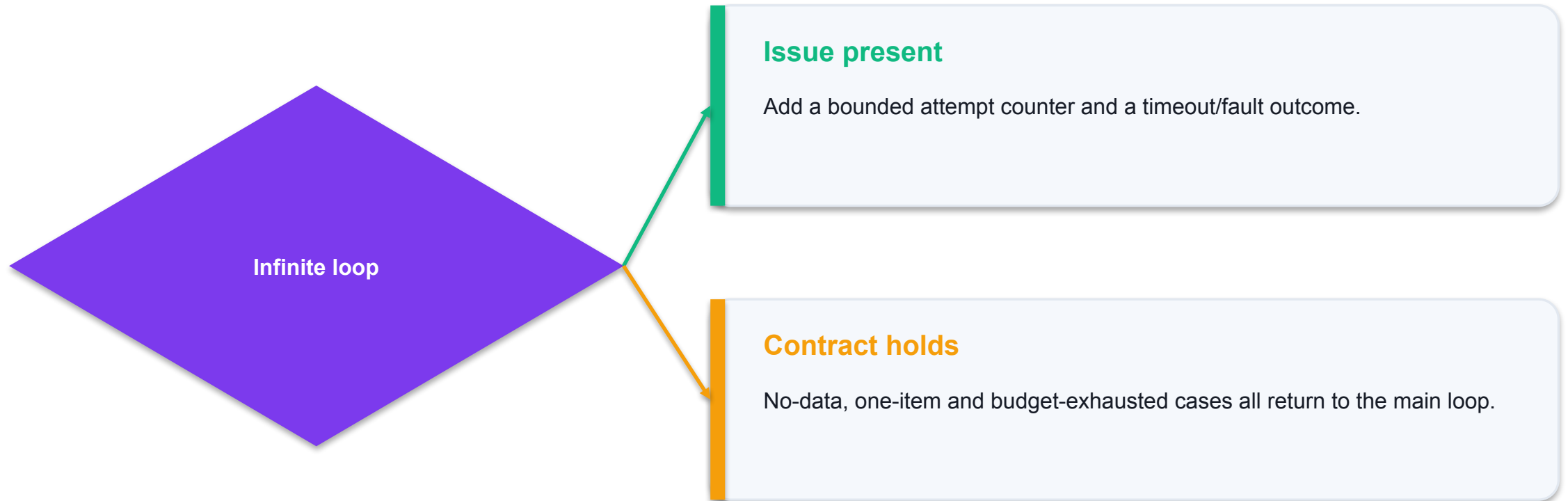
An unbounded retry loop can starve control work.

While and do-while differ at entry / contract

Contract field	Exact rule
Transport loop	Bound processing by bytes/time per main-loop pass.
No data	while must perform no read.
Retry policy	Maximum attempts and timeout are explicit.
Scheduler	Return control rather than wait indefinitely.

WHEN IT MATTERS No-data, one-item and budget-exhausted cases all return to the main loop.

While and do-while differ at entry / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Pointers require ownership and length

Exact mechanism / inspect the contract, not just syntax

```
void ring_push(Ring *r,int value);  
int ring_mean(const Ring *r);  
/* r points to a valid caller-owned Ring */  
/* const blocks writes through this pointer */
```

Ownership

The caller allocates and retains the Ring state.

Mutation

ring_push changes the pointed-to object.

Const

ring_mean promises not to modify through its parameter.

Lifetime

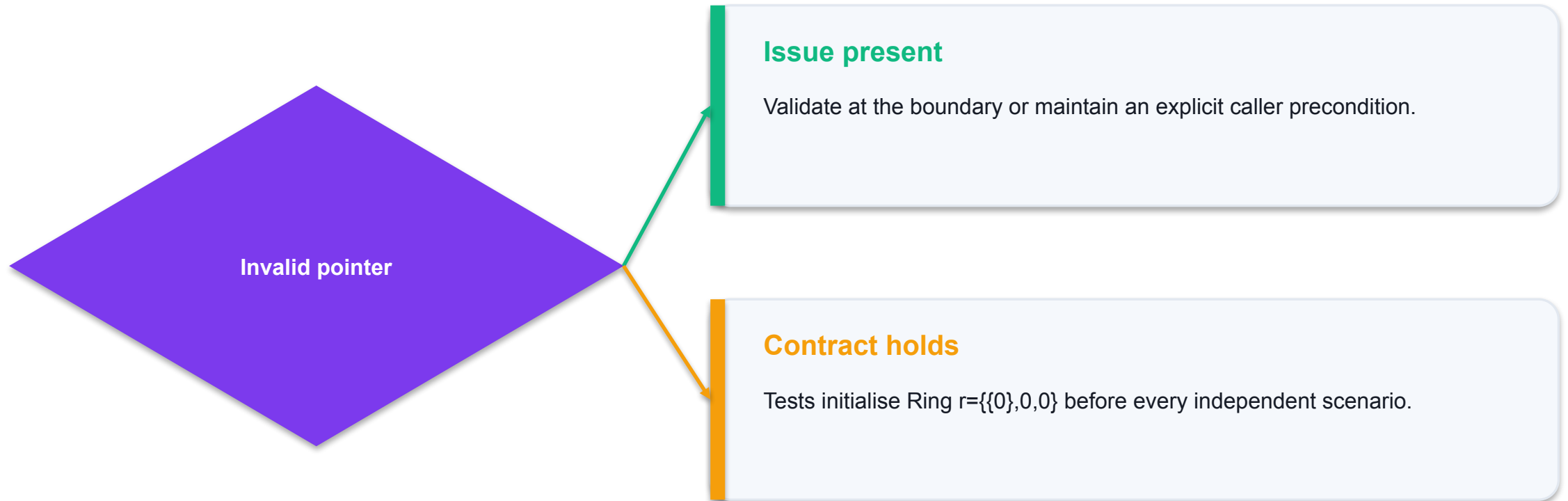
The state must outlive each call.

Pointers require ownership and length / contract

Contract field	Exact rule
Precondition	Pointer non-null and points to a valid initialised Ring.
Aliasing	Other writers must not corrupt count/next.
Array length	Length is not inferred from a bare pointer.
Reset	Initialise values, next and count before use.

WHEN IT MATTERS Tests initialise Ring $r=\{\{0\},0,0\}$ before every independent scenario.

Pointers require ownership and length / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Ring indexing keeps memory fixed

Exact mechanism / inspect the contract, not just syntax

```
r->values[r->next] = value;  
r->next = (r->next + 1u) % 4u;  
if (r->count < 4u) r->count++;
```

Index

next remains 0..3 after modulo.

Overwrite

The fifth sample replaces the oldest slot.

Capacity

No allocation occurs as samples arrive.

Count

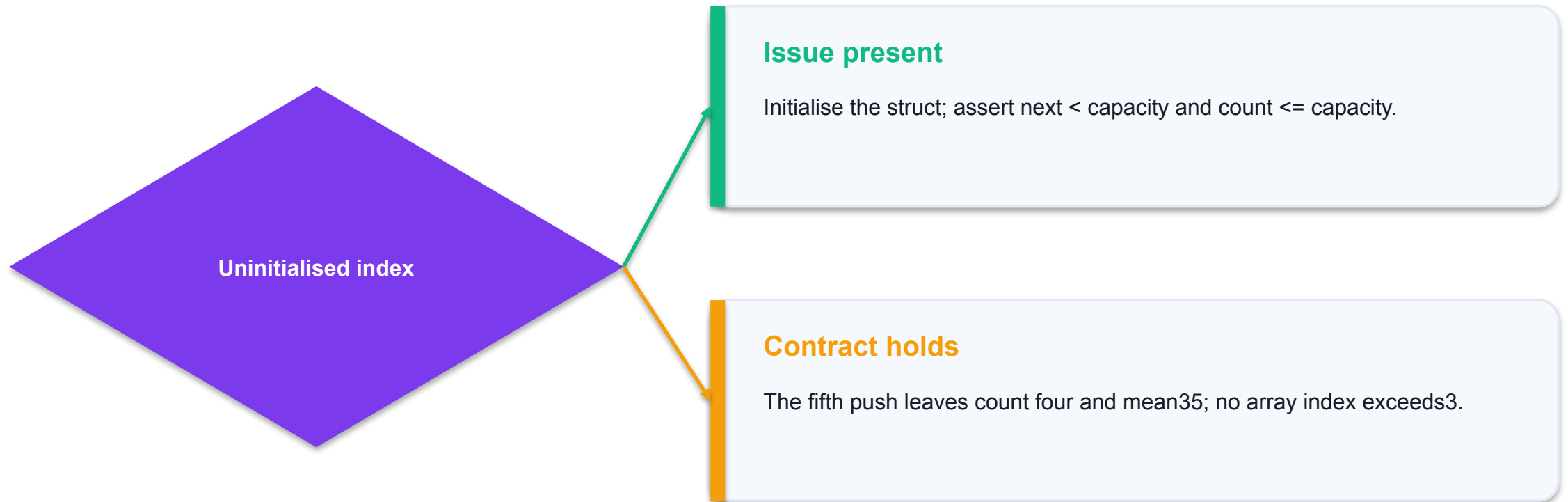
Count saturates at four, unlike next which wraps.

Ring indexing keeps memory fixed / contract

Contract field	Exact rule
Initial state	next=0,count=0.
After five pushes	next=1,count=4.
Sequence	10,20,30,40,50 retains 20,30,40,50.
Mean	35 with integer division.

WHEN IT MATTERS The fifth push leaves count four and mean35; no array index exceeds3.

Ring indexing keeps memory fixed / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Strings reserve a terminator byte

Exact mechanism / inspect the contract, not just syntax

```
char out[16];
int n = snprintf(out, sizeof out, "T=%d", 250);
if (n < 0 || (size_t)n >= sizeof out)
    return -1;
/* accepted text: T=250 */
```

Capacity

16 bytes includes the NUL terminator.

Return

snprintf reports the would-be length excluding NUL.

Truncation

$n \geq \text{capacity}$ is rejection even if output looks partly correct.

Comparison

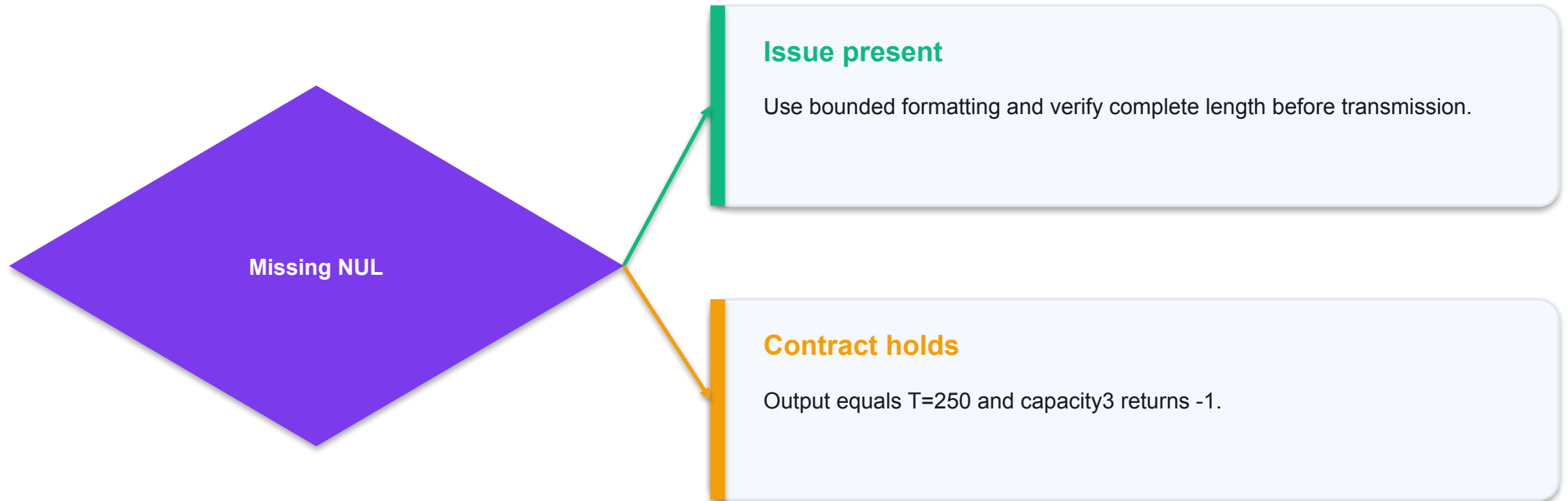
strcmp compares text, not pointer addresses.

Strings reserve a terminator byte / contract

Contract field	Exact rule
Successful length	5 for T=250.
Required capacity	At least 6 for that exact text.
Short capacity	3 must return truncation failure.
Caller	Never use truncated text as an accepted command.

WHEN IT MATTERS Output equals T=250 and capacity 3 returns -1.

Strings reserve a terminator byte / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Structs package values, not wire bytes

Exact mechanism / inspect the contract, not just syntax

```
typedef struct {  
    int temp_tenths;  
    int humidity;  
    uint32_t stamp;  
} Sample;  
/* serialize named fields explicitly */
```

Grouping

Fields belong to one sample contract.

Unit

Temperature and humidity use distinct units.

Padding

Compiler may insert bytes between fields.

Transport

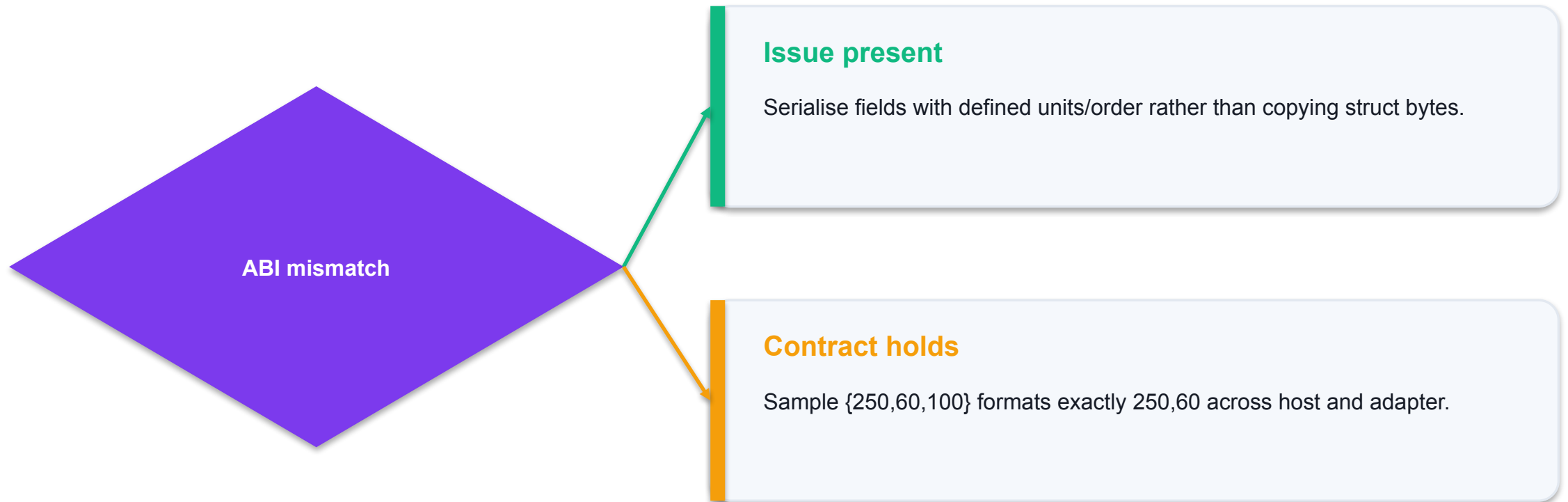
Raw `sizeof(Sample)` is not a portable serial schema.

Structs package values, not wire bytes / contract

Contract field	Exact rule
Temperature	-400..1250 tenths.
Humidity	0..100 percent.
Timestamp	uint32_t milliseconds.
CSV	Explicit field order temp_tenths,humidity.

WHEN IT MATTERS Sample {250,60,100} formats exactly 250,60 across host and adapter.

Structs package values, not wire bytes / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Unions and bitfields need a tag

Exact mechanism / inspect the contract, not just syntax

```
typedef enum { VALUE_INT, VALUE_FLOAT } Kind;
typedef struct {
    Kind kind;
    union { int whole; float fractional; } value;
} Reading;
/* read the member selected by kind */
```

Union

Members share storage; only the active representation is meaningful.

Tag

kind records which member the caller may interpret.

Bitfield

Layout and ordering are implementation-defined.

Portable status

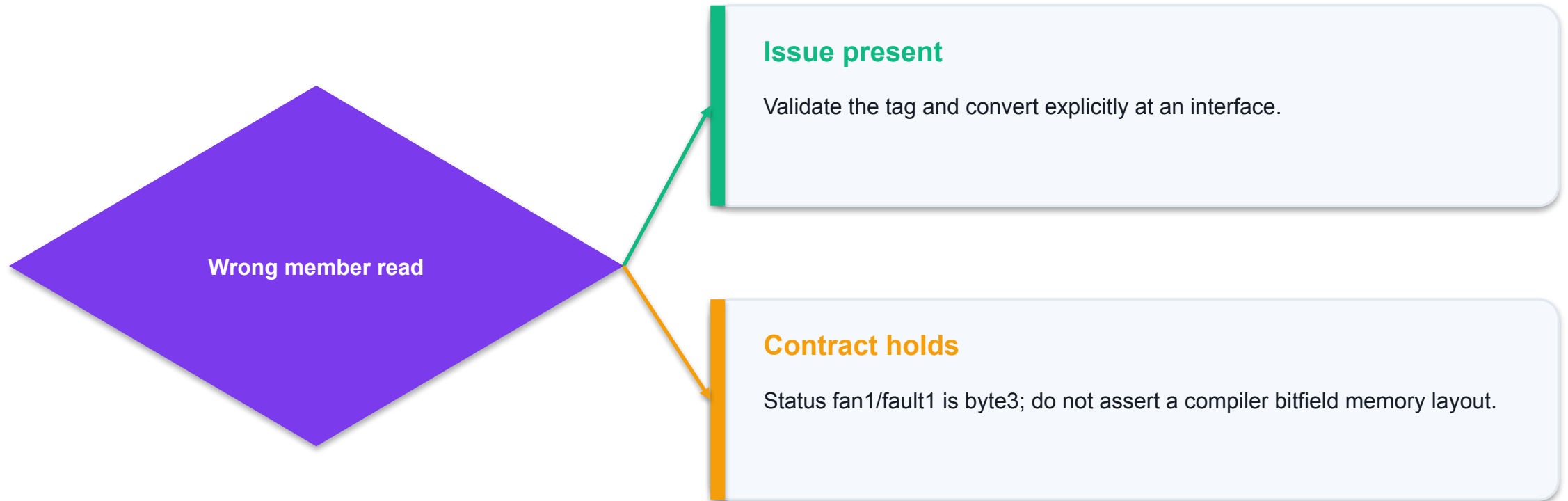
Use masks 1u and 2u for a defined wire byte.

Unions and bitfields need a tag / contract

Contract field	Exact rule
Tagged value	Check kind before interpreting value.
Unknown tag	Reject rather than reinterpret bytes.
Bit masks	bit0 fan; bit1 fault; remaining bits zero.
typedef	Names an existing type; does not allocate an object.

WHEN IT MATTERS Status fan1/fault1 is byte3; do not assert a compiler bitfield memory layout.

Unions and bitfields need a tag / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Implement fan modes and hysteresis

ACTIVITY 7

Independent progressive practice /full procedures in guide

A server-room fan has AUTO, MANUAL_ON and OFF modes. Temperature noise around the threshold must not chatter the output.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Seven assertions prove threshold boundaries, retention and unknown-mode OFF.

Implement fan modes and hysteresis / runtime



EXACT CONTRACT

Temperature uses tenths of degree; AUTO on at ≥ 300 , off at ≤ 280 , retained state between. Unknown mode fails OFF.

Implement fan modes and hysteresis / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int fan_next(int mode, int temp_tenths, int previous) {
    switch (mode) {
        case 0: if (temp_tenths>=300) return 1;
                if (temp_tenths<=280) return 0;
                return previous != 0;
        case 1: return 1;
        case 2: return 0;
        default: return 0;
    }
}
```

switch

Mode chooses the control strategy.

Hysteresis

Two thresholds create stateful deadband.

Units

300 means 30.0 degrees, not ADC counts.

Fail-safe

Unknown mode explicitly disables actuation.

Implement fan modes and hysteresis / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Temperature uses tenths of degree; AUTO on at ≥ 300 , off at ≤ 280 , retained state between. Unknown mode fails OFF. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Implement fan modes and hysteresis. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Implement fan modes and hysteresis / acceptance

✓ Expected result

Seven assertions prove threshold boundaries, retention and unknown-mode OFF. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Use an array-backed ring buffer

ACTIVITY 8

Independent progressive practice /full procedures in guide

Store the latest four environmental samples with fixed memory and compute their integer mean using a pointer to state.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Six assertions pass; fifth sample overwrites 10 and mean is 35.

Use an array-backed ring buffer / runtime



EXACT CONTRACT

Buffer capacity is 4; count 0..4; next 0..3; oldest sample overwritten. Empty mean returns 0. Samples are restricted to -1000..1000.

Use an array-backed ring buffer / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
void ring_push(Ring *r, int value) {
    if(value < -1000 || value > 1000) return;
    r->values[r->next] = value; r->next=(r->next+1u)%4u;
    if(r->count<4u) r->count++;
}
int ring_mean(const Ring *r) {
    int sum=0; if(r->count==0) return 0;
    for(unsigned i=0;i<r->count;i++) sum+=r->values[i];
    return sum/(int)r->count;
}
```

Pointer

Mutate caller state through Ring *.

Array

Indices remain inside four allocated elements.

Bounds

Sample restriction keeps int sum safe on 16-bit hosts too.

Mean

Integer division truncates; document rounding policy.

Use an array-backed ring buffer / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Buffer capacity is 4; count 0..4; next 0..3; oldest sample overwritten. Empty mean returns 0. Samples are restricted to -1000..1000. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Use an array-backed ring buffer. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Use an array-backed ring buffer / acceptance

✓ Expected result

Six assertions pass; fifth sample overwrites 10 and mean is 35. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Encode bounded status records

ACTIVITY 9

Independent progressive practice /full procedures in guide

Create a compact status byte and a human-readable record without serialising compiler-dependent struct memory.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Six assertions pass; status 3 sets only fan and fault bits; short buffer rejects truncation.

Encode bounded status records / runtime



EXACT CONTRACT

Temperature text fits caller capacity; sprintf truncation returns -1. Status bit 0=fan, bit 1=fault; other bits zero.

Encode bounded status records / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
#include <stdio.h>
uint8_t status_flags(int fan,int fault) {
    return (uint8_t)((fan?1u:0u) | (fault?2u:0u));
}
int status_text(char *out,size_t cap,int temp) {
    int n=snprintf(out,cap,"T=%d",temp);
    return n<0 || (size_t)n>=cap ? -1 : n;
}
```

String

Capacity includes terminating NUL.

Mask

Explicit bit positions define a portable byte contract.

Bitfield

C bitfield layout is not a portable wire protocol.

Truncation

A partially written string is not success.

Encode bounded status records / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Temperature text fits caller capacity; snprintf truncation returns -1. Status bit 0=fan, bit 1=fault; other bits zero. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Encode bounded status records. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Encode bounded status records / acceptance

✓ Expected result

Six assertions pass; status 3 sets only fan and fault bits; short buffer rejects truncation. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

TOPIC 4 /K4/A4

Contracts, interoperability and fault evidence

Activities 10-12 / specifications, contracts, failure and evidence

Serial framing precedes parsing

Exact mechanism / inspect the contract, not just syntax

```
bytes -> fixed-capacity receive buffer  
newline -> complete message  
overflow -> discard until newline  
complete line -> parse_led()  
accepted -> bounded LED request
```

Framing

A parser needs a complete NUL-terminated message.

Capacity

Reserve one byte for NUL and reject overflow.

Line ending

Transport removes newline/CR according to the contract.

Isolation

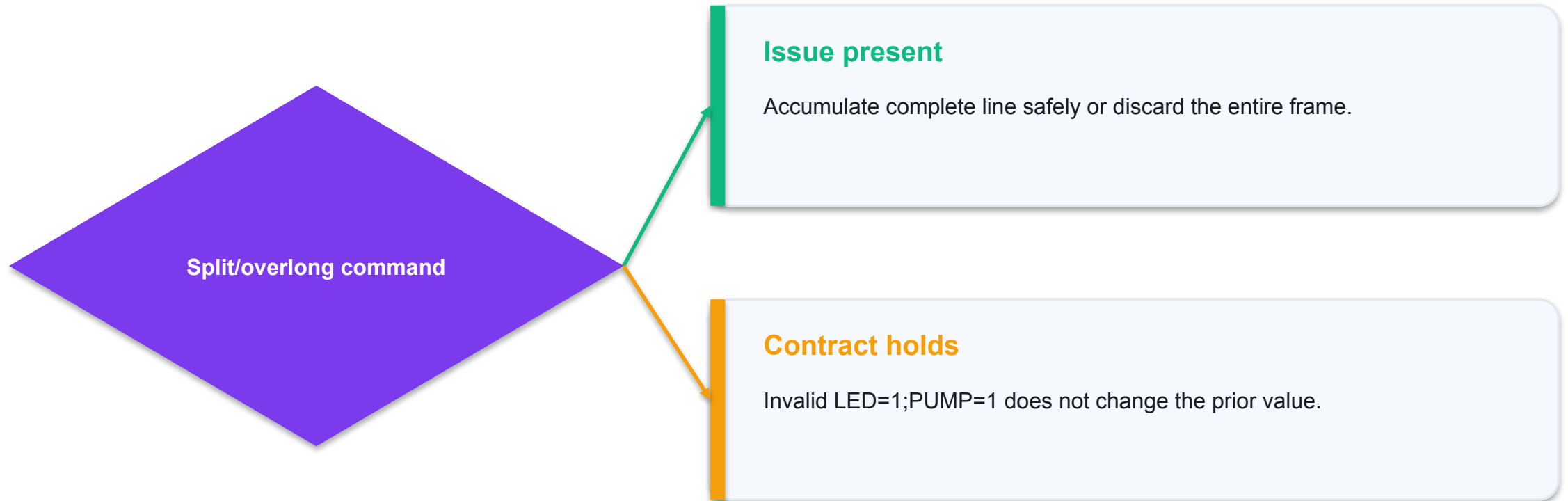
An invalid frame never partially controls an output.

Serial framing precedes parsing / contract

Contract field	Exact rule
Legal lines	LED=0 and LED=1 only.
Forbidden	Spaces, extra fields and semicolon chaining.
Atomic failure	Previous state remains unchanged.
Budget	Process a bounded number of bytes per loop.

WHEN IT MATTERS Invalid LED=1;PUMP=1 does not change the prior value.

Serial framing precedes parsing / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

An exact parser rejects extra tokens

Exact mechanism / inspect the contract, not just syntax

```
if (strcmp(line,"LED=0")==0) {  
    *value=0; return 1;  
}  
if (strcmp(line,"LED=1")==0) {  
    *value=1; return 1;  
}  
return 0;
```

Equality

Whole-string matching avoids accepting a dangerous prefix.

Allowlist

Only two exact commands are valid.

Return

1 accepted;0 rejected.

State

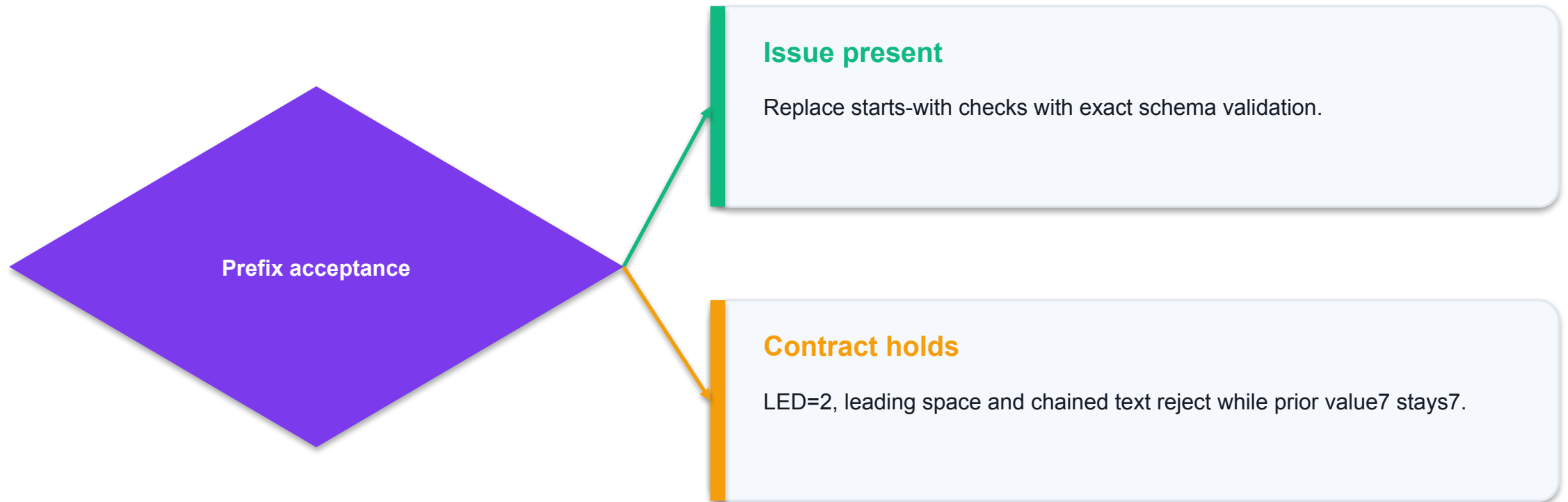
No assignment occurs on rejection.

An exact parser rejects extra tokens / contract

Contract field	Exact rule
Input	Valid NUL-terminated bounded string.
Output pointer	Valid caller-owned int.
Accepted	Sets 0 or 1.
Rejected	Leaves existing output unchanged.

WHEN IT MATTERS LED=2, leading space and chained text reject while prior value7 stays7.

An exact parser rejects extra tokens / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Freshness is part of interoperability

Exact mechanism / inspect the contract, not just syntax

```
uint32_t age = now - sample.stamp;  
int fresh = age <= 5000u;  
int units_ok = sample.temp_tenths >= -400  
               && sample.temp_tenths <= 1250;  
/* display/log only validated sample */
```

Age

Timestamp measures sample freshness, not receipt time alone.

Rollover

Unsigned subtraction supports wrap in the bounded-age context.

Unit range

Impossible converted values are rejected.

Consumer

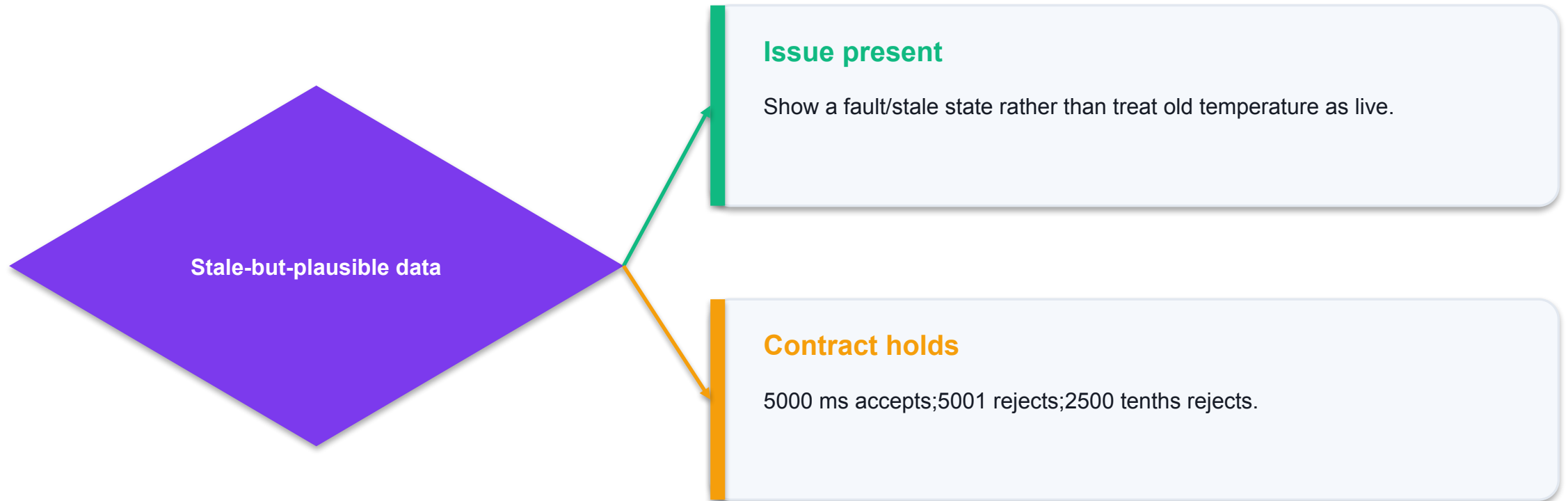
Display and logger need the same validation contract.

Freshness is part of interoperability / contract

Contract field	Exact rule
Fresh	Age0..5000 ms.
Stale	Age5001 ms and above.
Temperature	-400..1250 tenths.
Humidity	0..100 percent.

WHEN IT MATTERS 5000 ms accepts;5001 rejects;2500 tenths rejects.

Freshness is part of interoperability / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

I2C contracts include electrical levels

Exact mechanism / inspect the contract, not just syntax

```
MCU I2C -> address -> peripheral  
A4/A5: pullups to 3.3 V only  
Qwiic: I2C4 PD13/PD12  
LCD address: check actual module  
shared bus: unique addresses and safe levels
```

Bus

Matching function names do not prove matching physical connections.

Address

Module variants can use different addresses.

Voltage

Pull-ups determine bus high level.

Driver

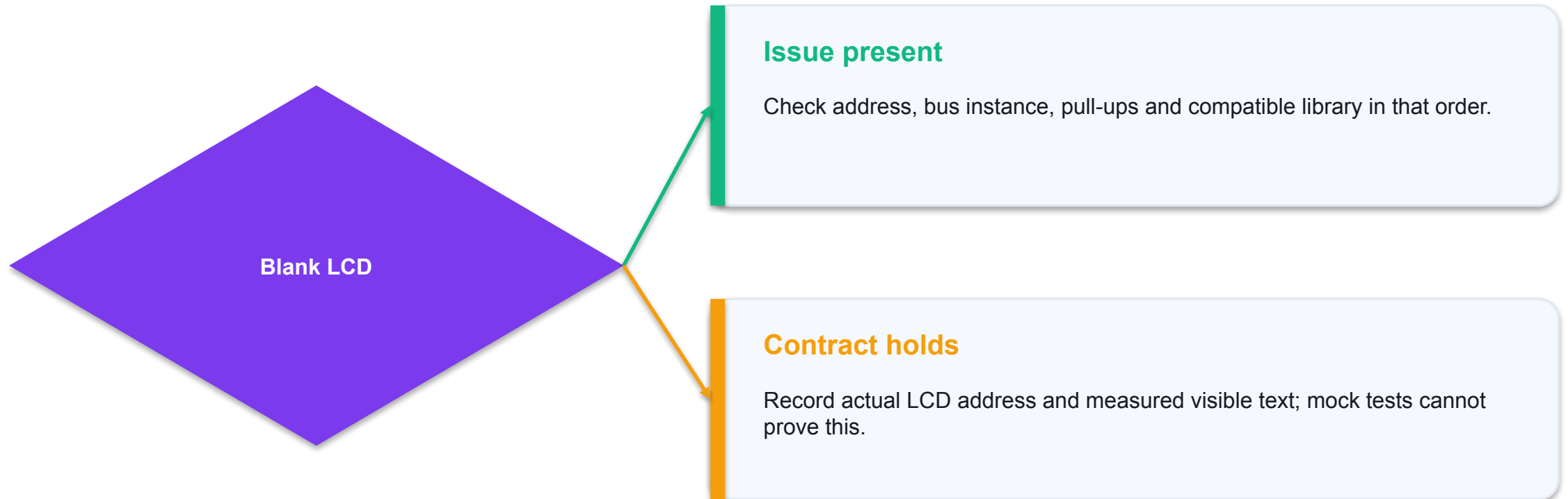
Installed library must support the selected bus instance.

I2C contracts include electrical levels / contract

Contract field	Exact rule
Before connection	Read peripheral voltage/interface manual.
Address evidence	Scan only on correctly reviewed wiring.
Display test	Write short known text and inspect rendering.
Interoperability	Document detection and fix, not just final success.

WHEN IT MATTERS Record actual LCD address and measured visible text; mock tests cannot prove this.

I2C contracts include electrical levels / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

SD logging needs write and readback

Exact mechanism / inspect the contract, not just syntax

```
validated Sample -> explicit CSV  
logger adapter -> selected SD library  
write result -> close/flush  
readback -> compare field values  
storage fault -> record safe degraded state
```

Serialisation

Write units and field order in the format spec.

Result

A call returning success is not sufficient without readback.

Resource

Bus/chip-select and library version must match hardware.

Failure

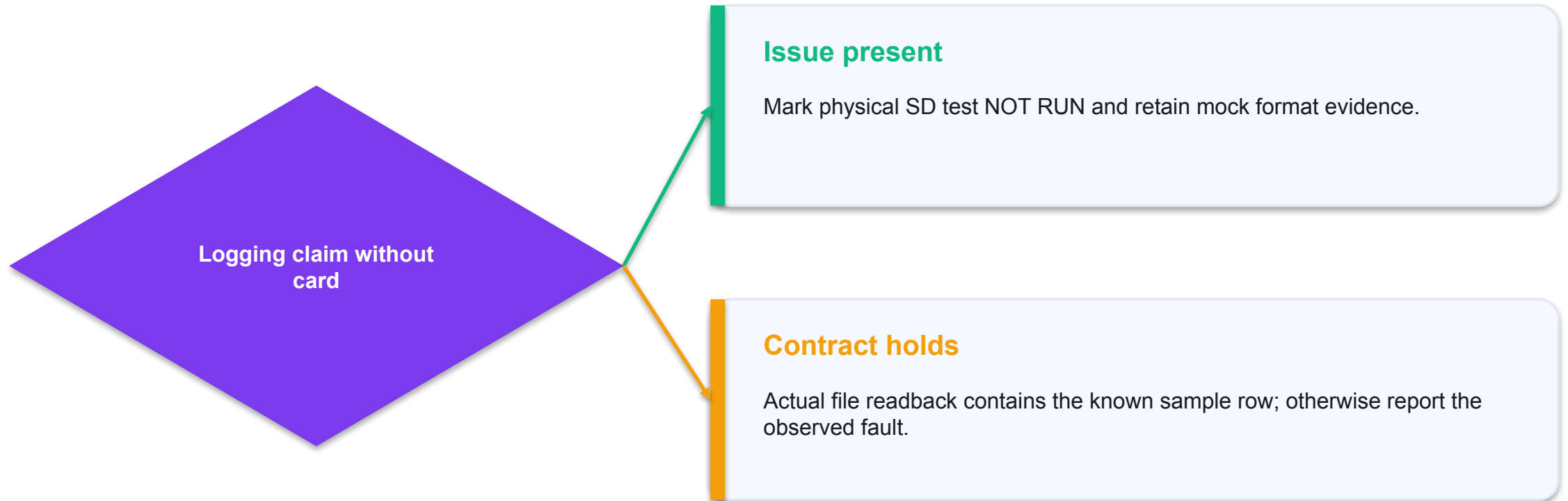
Storage loss must not freeze sensor/control work.

SD logging needs write and readback / contract

Contract field	Exact rule
Schema	temp_tenths,humidity in the host mock.
Target API	Verified selected-core SD library only.
Integrity	Check rows/values and malformed data.
Evidence	Host CSV formatting is distinct from actual card access.

WHEN IT MATTERS Actual file readback contains the known sample row; otherwise report the observed fault.

SD logging needs write and readback / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Fault gates must fail deterministically

Exact mechanism / inspect the contract, not just syntax

```
if (sensor_ok != 1) return 0;  
if (moisture < 0 || moisture > 100) return 0;  
if (age_ms > 2000u) return 0;  
return moisture < 30;
```

Fault

Disconnected sensor suppresses request even when dry.

Range

Invalid percentage cannot trigger actuation.

Timeout

Freshness is an independent condition.

Limit

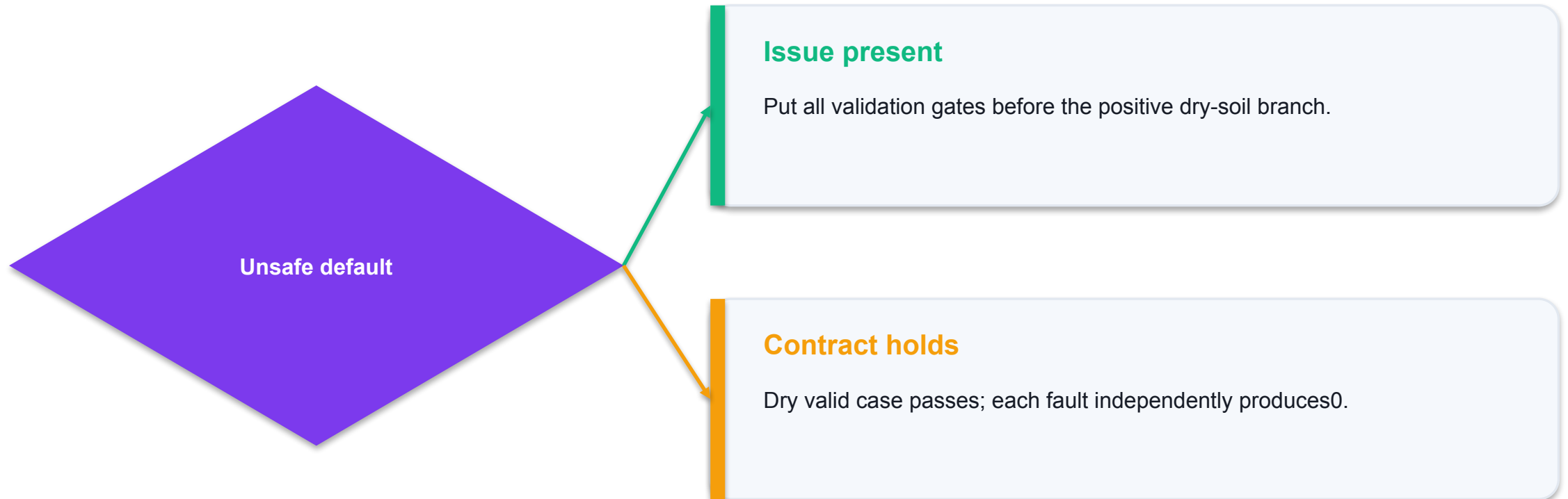
Pump maximum runtime requires another safety component.

Fault gates must fail deterministically / contract

Contract field	Exact rule
Dry	29 percent requests ₁ if valid/fresh.
Wet boundary	30 percent requests ₀ .
Stale boundary	2001 ms requests ₀ .
Invalid Boolean	sensor_ok ₂ requests ₀ .

WHEN IT MATTERS Dry valid case passes; each fault independently produces₀.

Fault gates must fail deterministically / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

A test oracle must be independent

Exact mechanism / inspect the contract, not just syntax

```
input: moisture29, age2000, ok1
requirement: dry AND fresh AND valid
expected: request1
input: moisture29, age2001, ok1
requirement: stale means safe-off
expected: request0
```

Oracle

Expected results come from specification, not from copying function output.

Boundary

Use neighbouring values around each inequality.

Isolation

Change one fault condition at a time.

Regression

Retain old tests when fixing a defect.

A test oracle must be independent / contract

Contract field	Exact rule
Positive test	Proves an allowed path works.
Negative test	Proves a forbidden path is blocked.
Mutation check	Change >2000 to >=2000 and observe boundary test failure.
Coverage limit	Passing examples do not prove all hardware safety.

WHEN IT MATTERS At2000 ms valid dry input1; at2001 ms0 catches the off-by-one mutation.

A test oracle must be independent / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Functionality indicators are observable

Exact mechanism / inspect the contract, not just syntax

```
functional: correct output for input
timing: response within contract
data: unit/range/schema consistent
resource: bounded memory/work
fault: safe rejection and recovery
evidence: repeatable readback
```

Correctness

The expected output must match the user requirement.

Timing

Host due checks are not physical response-time measurements.

Interoperability

Every producer/consumer agrees on units and framing.

Recovery

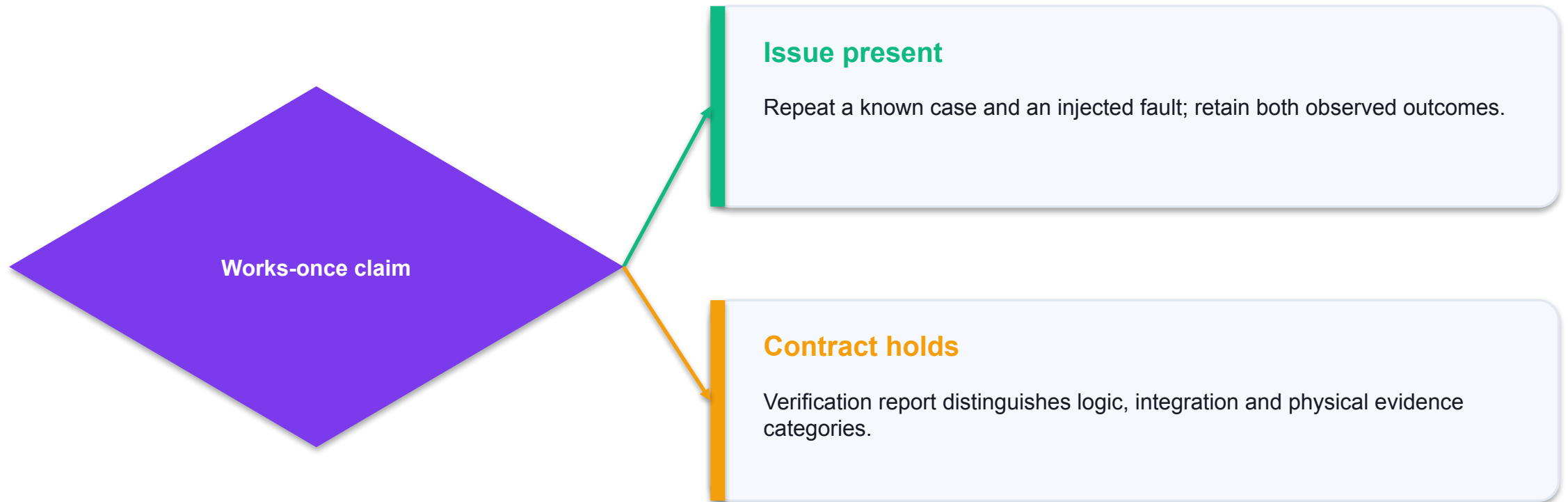
A fault must not permanently block the main loop.

Functionality indicators are observable / contract

Contract field	Exact rule
Host evidence	Assertions, warning gate, deterministic fixtures.
Integration evidence	Known sample passes through display/log mocks.
Physical evidence	Board timestamps, visible output and storage readback.
Issue report	Symptom, cause, fix and retest for at least two issues.

WHEN IT MATTERS Verification report distinguishes logic, integration and physical evidence categories.

Functionality indicators are observable / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Bridge RPC is a value boundary

Exact mechanism / inspect the contract, not just syntax

```
#include <Arduino_RouterBridge.h>
Bridge.begin();
Monitor.begin(115200);
Bridge.provide_safe("set_permit", set_permit);
// Python: Bridge.call("set_permit", value, timeout=1.0)
// MCU Bridge.call returns async RpcCall; result(ref) blocks
```

MCU

provide_safe runs handlers safely with shared main-loop state; no nested RPC call in a callback.

Python

from arduino.app_utils import Bridge; call is synchronous and can raise timeout/connection/value errors.

Environment

APP_SOCKET and arduino-router/App Lab environment are prerequisites.

Safety

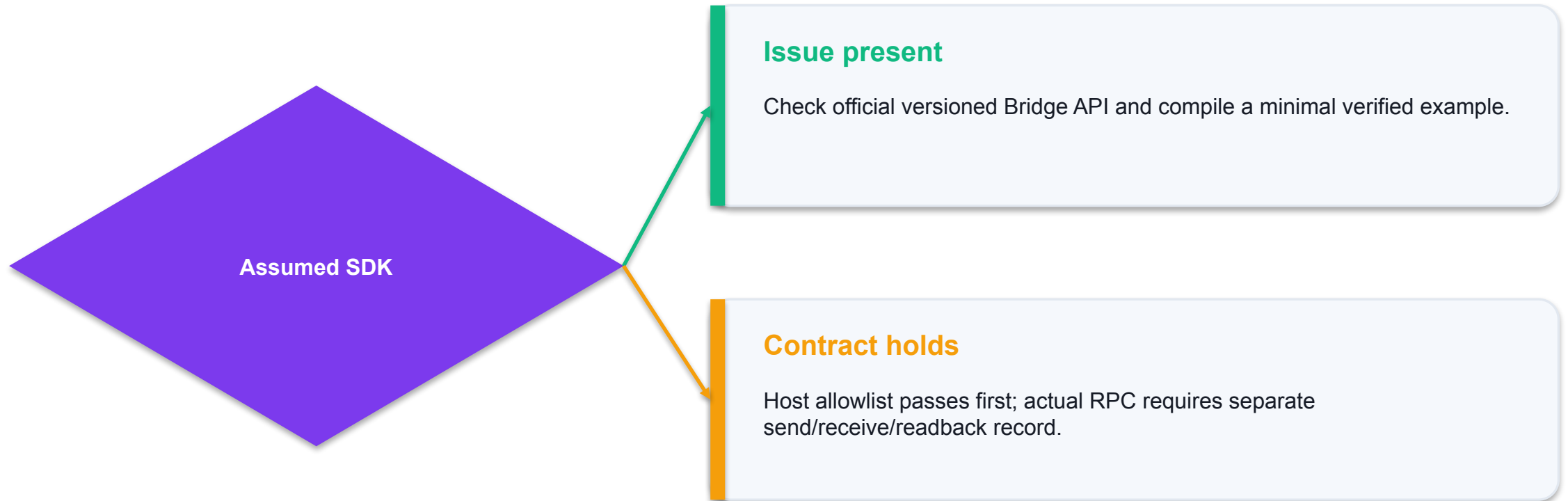
UART is routed by Zephyr; notify has no response and must not authorise actuation.

Bridge RPC is a value boundary / contract

Contract field	Exact rule
Python timeout	Explicit 1.0s, not default 10s; reject timeout/error.
MCU return	RpcCall.result(ref) is blocking, bool success, result read once.
Handler policy	Validate 0/1 and use a TTL lease in deterministic MCU logic.
Physical evidence	Host mock is not proof of actual RPC send/receive/readback.

WHEN IT MATTERS Host allowlist passes first; actual RPC requires separate send/receive/readback record.

Bridge RPC is a value boundary / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Parse a bounded serial command

ACTIVITY 10

Independent progressive practice /full procedures in guide

Separate serial transport from a parser that accepts only LED=0 or LED=1. A model response must never become arbitrary actuator code.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Five assertions pass; invalid text preserves the previous output.

Parse a bounded serial command / runtime



EXACT CONTRACT

Input is NUL-terminated exact text, line terminator removed by transport. Parser does not accept spaces, extra tokens or unbounded commands.

Parse a bounded serial command / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
#include <string.h>
int parse_led(const char *line,int *value) {
    if(strcmp(line,"LED=0")==0) { *value=0; return 1; }
    if(strcmp(line,"LED=1")==0) { *value=1; return 1; }
    return 0;
}
```

Framing

Transport supplies one complete bounded line.

Allowlist

Exactly two commands are legal.

Atomicity

Invalid input leaves previous value unchanged.

Injection

Semicolon chaining is rejected as a whole.

Parse a bounded serial command / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Input is NUL-terminated exact text, line terminator removed by transport. Parser does not accept spaces, extra tokens or unbounded commands. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Parse a bounded serial command. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Parse a bounded serial command / acceptance

✓ Expected result

Five assertions pass; invalid text preserves the previous output. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Integrate sensor display and logging

ACTIVITY 11

Independent progressive practice /full procedures in guide

Sensor, display and logger modules exchange one explicit sample struct. Detect unit mismatch and stale timestamps before display or logging.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Seven assertions pass; 5000 ms valid, 5001 ms stale; 2500 and humidity 101 rejected.

Integrate sensor display and logging / runtime



EXACT CONTRACT

Sample temperature is tenths of degree, humidity percent 0..100, timestamp ms. Valid temperature -400..1250; stale after 5000 ms.

Integrate sensor display and logging / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
#include <stdio.h>
int sample_valid(const Sample *s,uint32_t now) {
    return s->temp_tenths>=-400 && s->temp_tenths<=1250 &&
    s->humidity>=0 && s->humidity<=100 &&
    (uint32_t)(now-s->stamp)<=5000u;
}
int sample_csv(char *out,size_t cap,const Sample *s) {
    int n=snprintf(out,cap,"%d,%d",s->temp_tenths,s->humidity);
    return n<0 || (size_t)n>=cap ? -1 : n;
}
```

Units

250 denotes 25.0 degrees in every module.

Freshness

Timestamp is part of the interface, not an optional comment.

Serialisation

CSV explicitly writes values, never sizeof(struct) bytes.

Hardware

LCD address and SD bus conflicts need physical tests.

Integrate sensor display and logging / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Sample temperature is tenths of degree, humidity percent 0..100, timestamp ms. Valid temperature -400..1250; stale after 5000 ms. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Integrate sensor display and logging. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Integrate sensor display and logging / acceptance

✓ Expected result

Seven assertions pass; 5000 ms valid, 5001 ms stale; 2500 and humidity 101 rejected. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Build a fault-injection gate

ACTIVITY 12

Independent progressive practice /full procedures in guide

An irrigation request must fail off for stale, disconnected or invalid sensing. Test a guard separately from the pump driver.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Seven assertions pass; each injected invalid/stale condition returns OFF.

Build a fault-injection gate / runtime



EXACT CONTRACT

Moisture 0..100 percent; sensor_ok 0/1. Output is ON only below 30 with age<=2000 ms and sensor_ok true; all other cases OFF.

Build a fault-injection gate / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int water_request(int moisture,unsigned age_ms,int sensor_ok) {
  if(sensor_ok!=1 || moisture<0 || moisture>100 || age_ms>2000u)
    return 0;
  return moisture<30;
}
```

Boundary

29 and 30 distinguish strict threshold.

Timeout

2001 ms is stale even if moisture is dry.

Fault

sensor_ok must be exactly 1.

Safety

This request gate does not implement maximum pump runtime.

Build a fault-injection gate / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Moisture 0..100 percent; sensor_ok 0/1. Output is ON only below 30 with age<=2000 ms and sensor_ok true; all other cases OFF. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Build a fault-injection gate. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Build a fault-injection gate / acceptance

✓ Expected result

Seven assertions pass; each injected invalid/stale condition returns OFF. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Irrigation / specification before actuation



Input

Moisture 0..100 percent plus freshness and sensor validity.

Threshold

Below 30 percent requests water only when valid and fresh.

Safety

A request is not pump approval; driver and runtime limit are separate.

Evidence

Fault injection must demonstrate an OFF request.

AI-generated conceptual scenario illustration; not a wiring plan or measured hardware evidence.

TOPIC 5 /K5/A5

User-aligned documentation and bounded extension

Activities 13-14 / specifications, contracts, failure and evidence

Trace requirements into maintenance docs

Exact mechanism / inspect the contract, not just syntax

```
R1 dry soil -> safe_request -> test29 -> ON
R2 wet soil -> safe_request -> test30 -> OFF
R3 fault/disable -> guard -> invalid tests -> OFF
R4 hardware timing -> board record -> NOT RUN
R5 maintenance -> header + diagram + report
```

Trace

Each user specification has an implementation and test reference.

Coverage

Untested physical requirements remain explicitly open.

Maintenance

A new developer can locate thresholds, units and interfaces.

AI

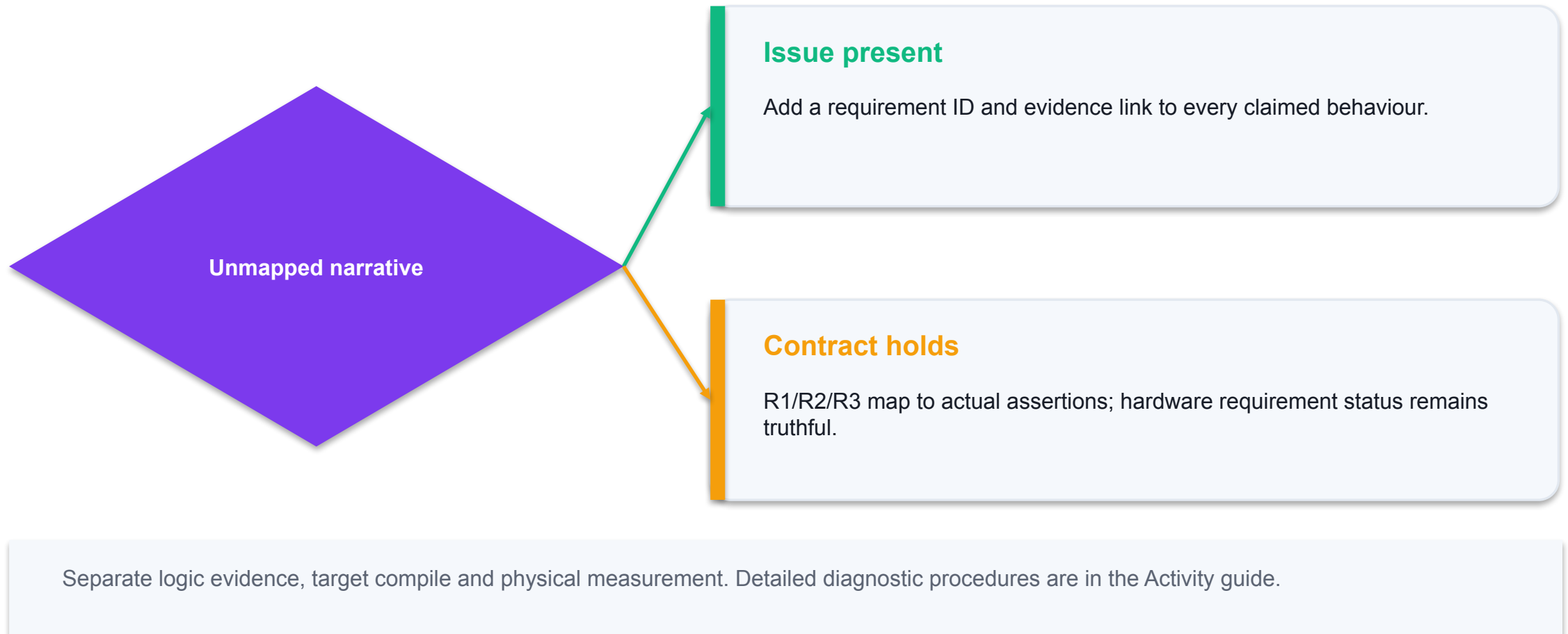
Generated prose is reviewed against code, not accepted as authority.

Trace requirements into maintenance docs / contract

Contract field	Exact rule
Annotated source	Why the guard exists and what units it expects.
Architecture	Data path, ownership and hardware boundary.
I/O specification	Ranges, polarity, timing and driver assumptions.
Project report	Evidence, known limits, changes and support.

WHEN IT MATTERS R1/R2/R3 map to actual assertions; hardware requirement status remains truthful.

Trace requirements into maintenance docs / failure



Architecture diagram is not a wiring plan

Exact mechanism / inspect the contract, not just syntax

```
soil sensor adapter -> percentage
percentage -> valid/fresh gate
gate -> request policy
request -> driver adapter
status -> LCD/log adapters
power safety -> separate approved circuit
```

Data flow

Arrows describe values and function boundaries.

Electrical

Voltage/current/wiring is documented separately.

Ownership

One module owns each input conversion and output driver.

Scenario

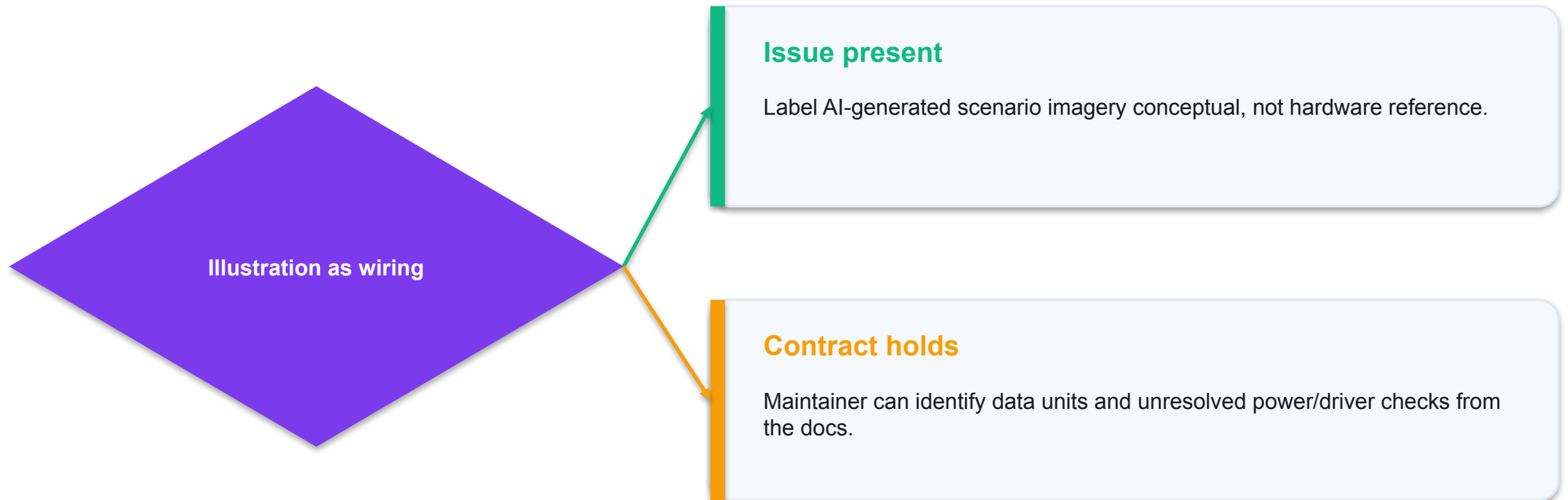
Illustration helps context but is not an exact board schematic.

Architecture diagram is not a wiring plan / contract

Contract field	Exact rule
Sensor input	Raw count and conversion setting.
Logic input	Percentage0..100 plus freshness.
Actuator output	Boolean request to a rated driver.
Physical review	Exact pinout and peripheral manuals are required.

WHEN IT MATTERS Maintainer can identify data units and unresolved power/driver checks from the docs.

Architecture diagram is not a wiring plan / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Local AI proposes; deterministic C decides

Exact mechanism / inspect the contract, not just syntax

```
optional local model -> mock/real proposal
proposal {value,confidence,age_ms}
policy: LED only, value0/1
confidence80..100, age<=1000
invalid -> reject (-1)
accepted -> bounded MCU request
```

Beginner scope

Extension reuses simple if logic and clear data contracts.

Local model

Model availability/speed are not assumed or measured here.

Safety

No shell, network, pump or arbitrary code commands.

Fallback

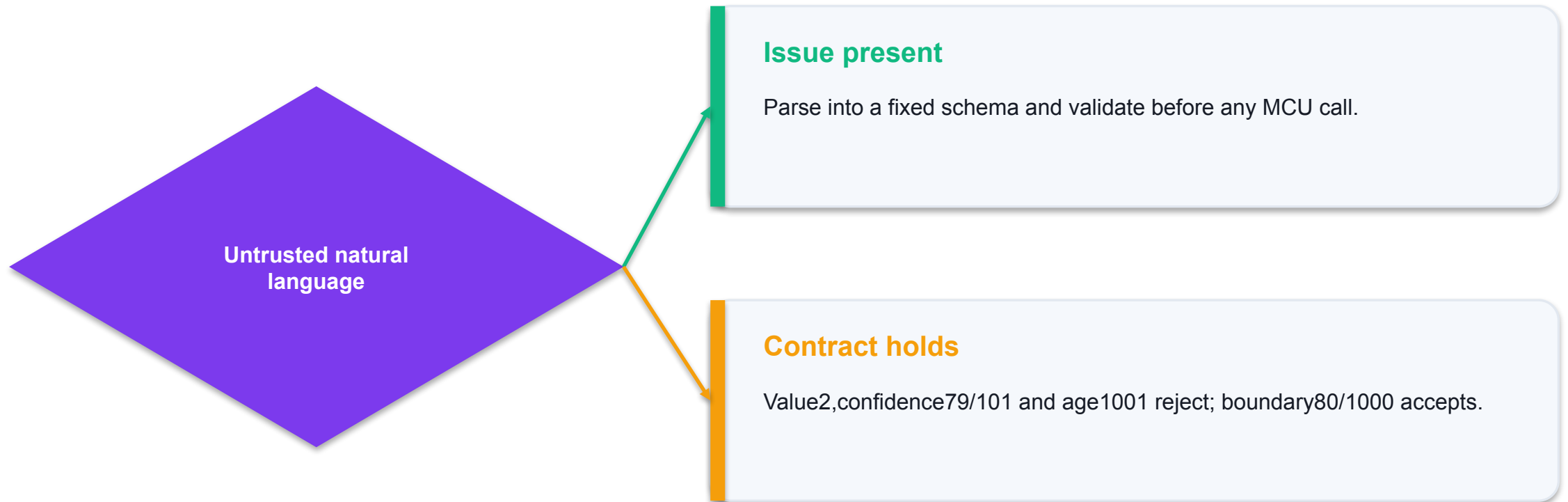
Mock data keeps the activity runnable without a model.

Local AI proposes; deterministic C decides / contract

Contract field	Exact rule
Value	Exactly 0 or 1.
Confidence	Teaching field 80..100; not calibrated accuracy.
Age	0..1000 ms.
Rejection	-1; no actuator change from an invalid proposal.

WHEN IT MATTERS Value 2, confidence 79/101 and age 1001 reject; boundary 80/1000 accepts.

Local AI proposes; deterministic C decides / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

User documentation needs honest limits

Exact mechanism / inspect the contract, not just syntax

```
Host logic tests: executed / pass count
MCU compile: recorded / not run
Board upload: recorded / not run
LCD and SD: observed / not run
Pump deployment: safety approval required
Local AI performance: not measured
```

Limit

Do not convert an unrun test into a success statement.

Separation

Compile, upload, function and safety approvals differ.

Support

Include reproduction steps and observed symptom.

Handover

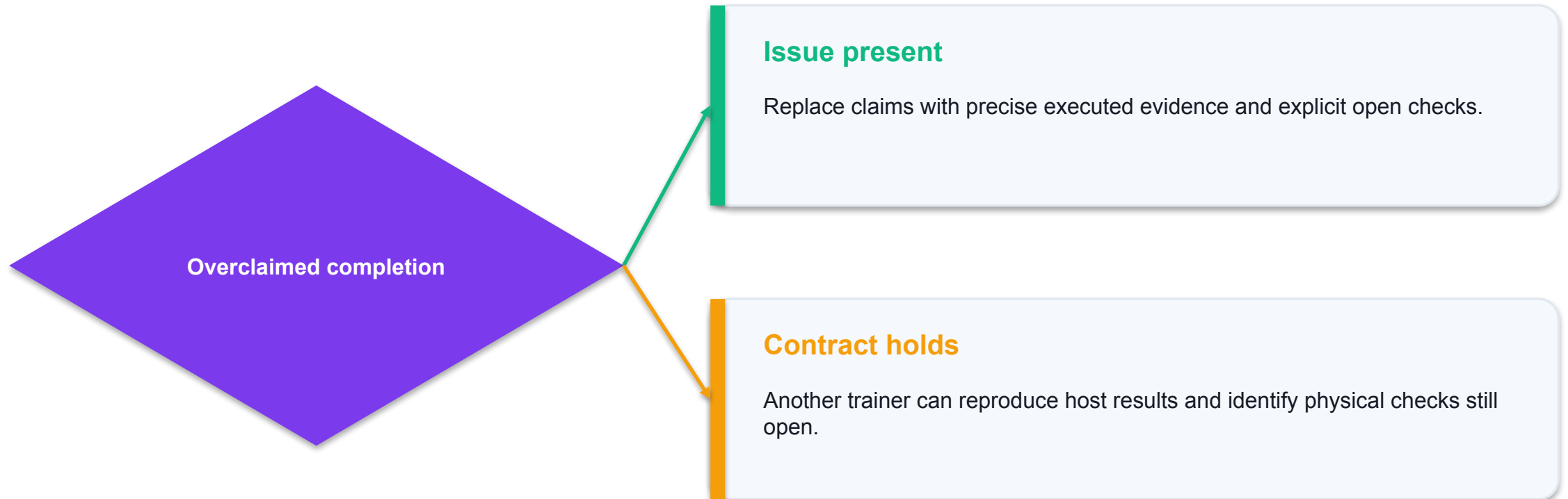
The client can see what remains to be verified.

User documentation needs honest limits / contract

Contract field	Exact rule
Record fields	Date, variant, revision, tools, fixture and result.
Failure record	Expected, actual, cause, fix and retest.
Privacy	No NRIC, credentials or private assessment answers in learner repo.
Release	Course version2.0 and dated change record.

WHEN IT MATTERS Another trainer can reproduce host results and identify physical checks still open.

User documentation needs honest limits / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Document changes preserve the contract

Exact mechanism / inspect the contract, not just syntax

```
/* proposed change: dry threshold30 ->35 */  
/* update requirement R1 */  
/* add tests34/35 */  
/* review pump safety impacts */  
/* regenerate docs and slide references */  
/* versioned reviewed release */
```

Change request

Threshold changes start in the specification.

Regression

Retain fault and timeout tests unchanged.

Traceability

Update code, header, diagram/report and acceptance together.

Release

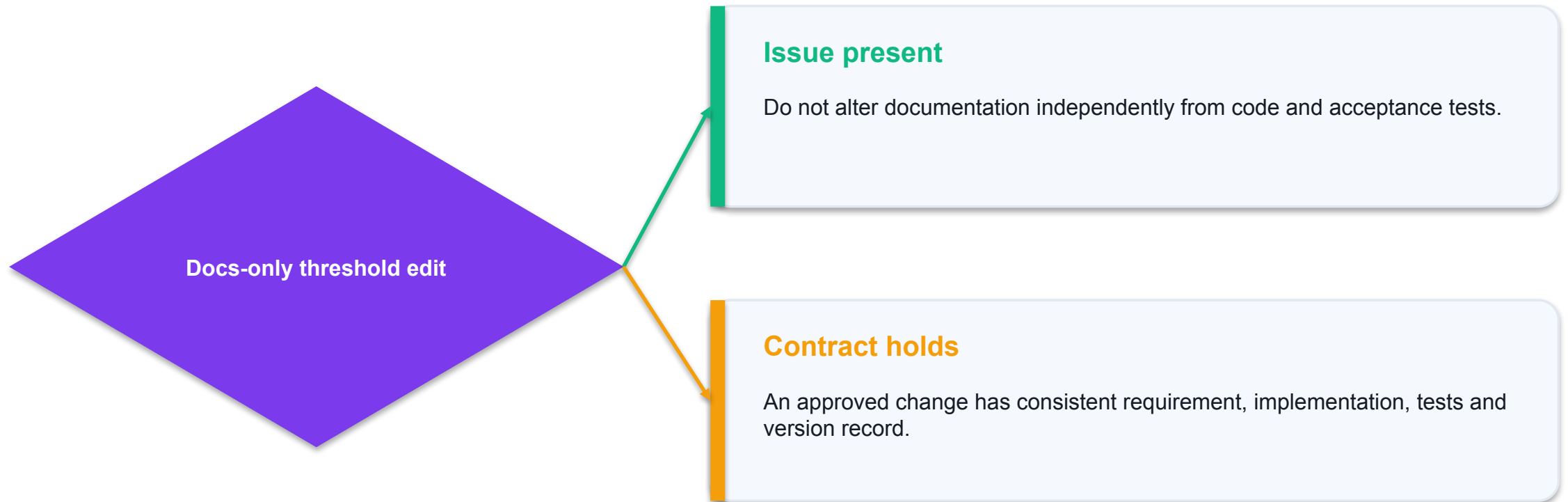
Slide references come from actual slide_map.json, not guessed numbers.

Document changes preserve the contract / contract

Contract field	Exact rule
Before change	29/30 test the original boundary.
After approved change	34/35 test the new boundary.
Unchanged safety	Invalid/stale input remains off.
Documentation	Date, reason, author and evidence are recorded.

WHEN IT MATTERS An approved change has consistent requirement, implementation, tests and version record.

Document changes preserve the contract / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Doxygen tags document the public interface

Exact mechanism / inspect the contract, not just syntax

```
/** @brief Validate a watering request.  
 * @param moisture Percentage0..100.  
 * @param age_ms Sample age in milliseconds.  
 * @return 1 only if dry,valid,fresh;else0.  
 * @note No GPIO side effects;not full pump safety.  
 */
```

Interface

Tags state units and exact return semantics.

Requirement

Add IDs linking user need to implementation and tests.

Generated docs

A documentation tool formats comments; it does not prove truth.

Review

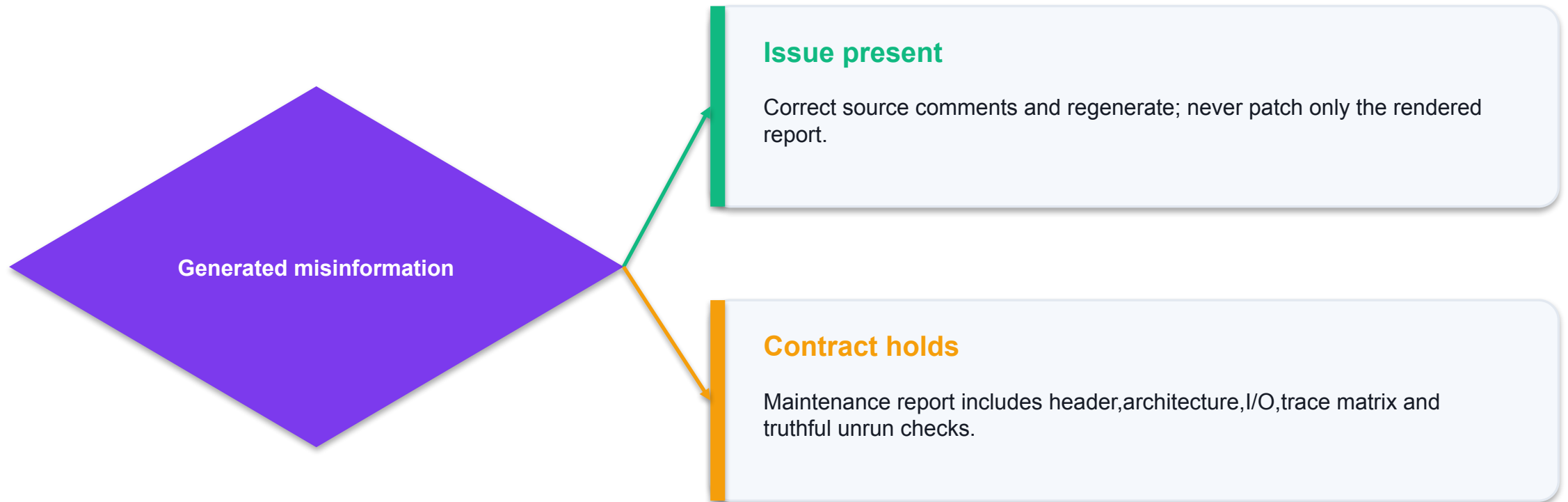
Compare every generated claim with code and executed evidence.

Doxygen tags document the public interface / contract

Contract field	Exact rule
Input	Annotated logic.h and logic.c.
Output	Readable function documentation and source links.
Validation	Check threshold and failure policies in generated text.
Optional tool	Doxygen installation is not needed for the host C gate.

WHEN IT MATTERS Maintenance report includes header,architecture,I/O,trace matrix and truthful unrun checks.

Doxygen tags document the public interface / failure



Separate logic evidence, target compile and physical measurement. Detailed diagnostic procedures are in the Activity guide.

Document an irrigation interface

ACTIVITY 13

Independent progressive practice /full procedures in guide

A maintenance team needs annotated headers, I/O specifications, a system diagram and a report tracing user requirements to tests.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Five assertions pass; documentation contains R1/R2/R3 trace rows and NOT RUN board fields until measured.

Document an irrigation interface / runtime



EXACT CONTRACT

Named requirement IDs R1/R2/R3 map to dry/wet/fault evidence. `safe_request` requires enabled and valid moisture; no hardware claims in documentation.

Document an irrigation interface / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
/* R1: dry soil requests water; R2: wet soil stays off.
 * R3: disabled/invalid sensing fails off. Unit: percent. */
int safe_request(int enabled,int moisture) {
    if(enabled!=1 || moisture<0 || moisture>100) return 0;
    return moisture<30;
}
```

Header

State units, ranges, return semantics and preconditions.

Traceability

Each requirement links to code and an assertion.

Diagram

Draw data path separately from power wiring.

Handover

List known limits and evidence still requiring a board.

Document an irrigation interface / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Named requirement IDs R1/R2/R3 map to dry/wet/fault evidence. safe_request requires enabled and valid moisture; no hardware claims in documentation. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Document an irrigation interface. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Document an irrigation interface / acceptance

✓ Expected result

Five assertions pass; documentation contains R1/R2/R3 trace rows and NOT RUN board fields until measured. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

Bound a UNO Q local-AI suggestion

ACTIVITY 14

Independent progressive practice /full procedures in guide

Optional extension: a Linux-side AI model suggests a low-power LED state. C-compatible policy validates value, confidence and age before any MCU request.

YOU'LL BUILD

C logic +host tests +verification record

TOOLCHAIN

Host C compiler /Arduino core /mock fixtures

DONE WHEN

Six assertions pass; all untrusted invalid or late proposals return -1.

Bound a UNO Q local-AI suggestion / runtime



EXACT CONTRACT

Proposal value is 0/1; confidence integer 0..100, minimum 80; age ≤ 1000 ms. Invalid suggestion returns -1 and must not reach an actuator.

Bound a UNO Q local-AI suggestion / C logic

Runnable reference extract /Arduino APIs remain in the C++ adapter

```
#include "logic.h"
int proposal_led(int value,int confidence,unsigned age_ms) {
    if((value!=0 && value!=1) || confidence<80 || confidence>100
    || age_ms>1000u) return -1;
    return value;
}
```

Dual brain

Linux produces a proposal; STM32 enforces deterministic policy.

Contract

Confidence is a mock field, not measured model accuracy.

Allowlist

LED only; pump, shell, file and network commands forbidden.

Failure

Timeout/model absence must leave output in safe default.

Bound a UNO Q local-AI suggestion / vibe-coding prompts

Design contract

- Design C-compatible logic for this exact contract: Proposal value is 0/1; confidence integer 0..100, minimum 80; age $\leq$ 1000 ms. Invalid suggestion returns -1 and must not reach an actuator. Keep Arduino APIs in the C++ adapter.

Vibe-code implementation

- Implement Bound a UNO Q local-AI suggestion. Preserve logic.h, units and thresholds. Explain every assumption. Add independently predicted boundary tests; do not invent hardware/API evidence.

Repair from evidence

- Given the first failed assertion, explain expected versus actual, identify the smallest contract-preserving fix and add a regression test. Do not edit expected values to hide the defect.

Bound a UNO Q local-AI suggestion / acceptance

✓ Expected result

Six assertions pass; all untrusted invalid or late proposals return -1. Physical-board checks remain NOT RUN until observed; use the detailed guide and record actual output separately.

IF IT DOESN'T WORK

Assertion fails

Compare exact inequality, unit and initial state with the fixture.

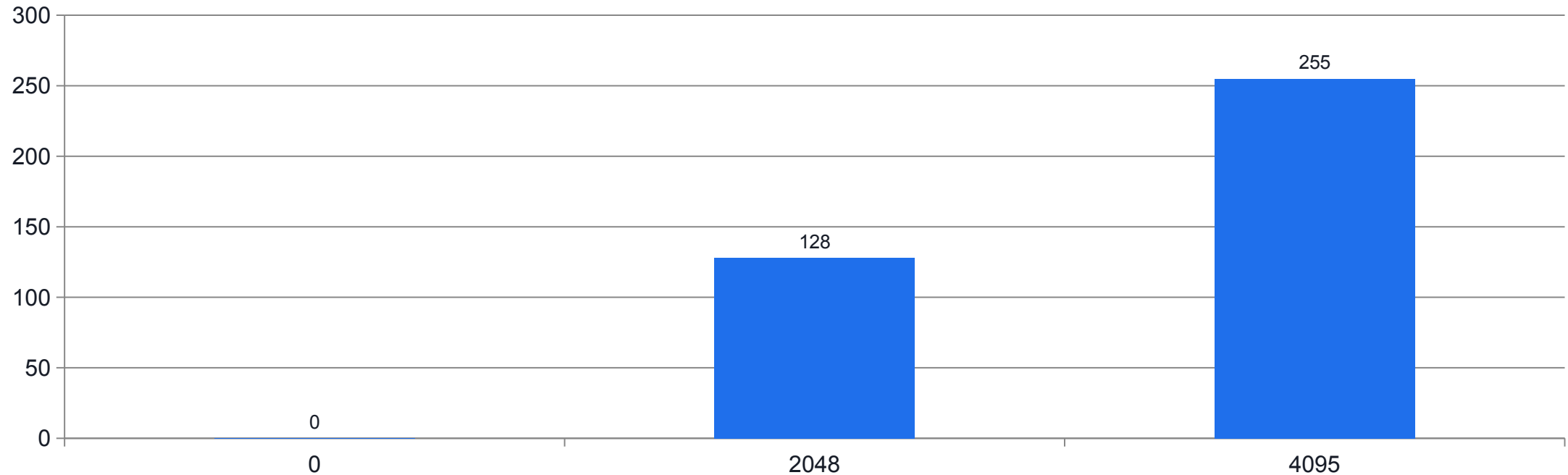
Target compile fails

Check selected board/core, header linkage and verified API.

Hardware not run

Record NOT RUN; host tests do not prove upload or physical actuation.

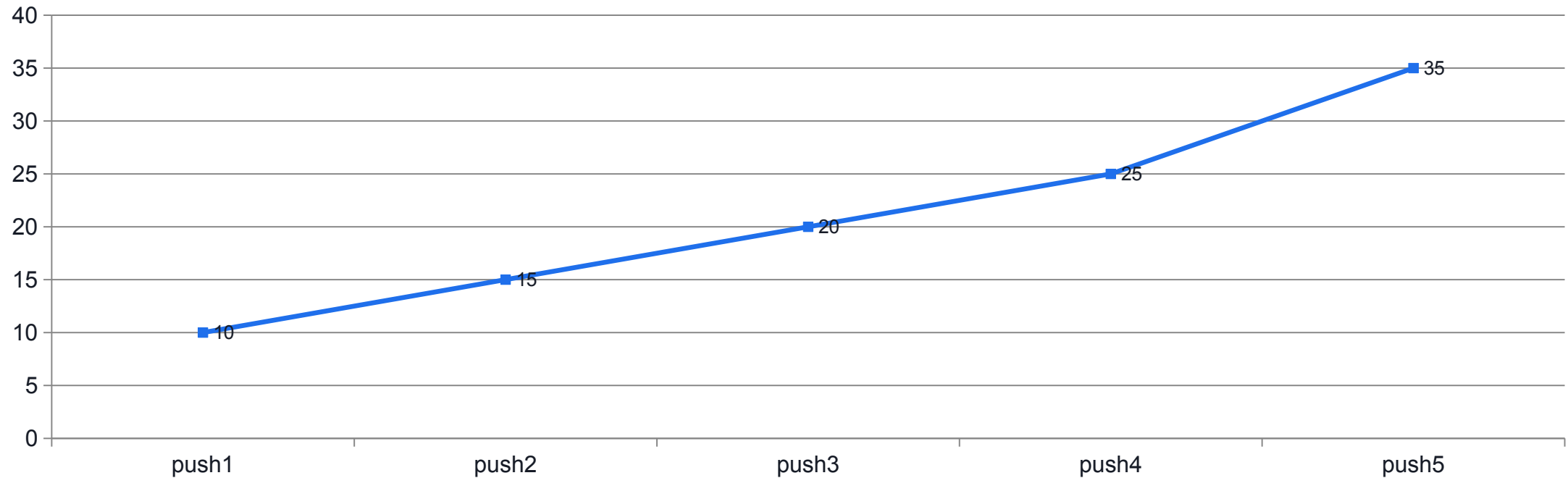
ADC scaling / deterministic fixture



WHAT THE DATA SHOWS

These are calculated host fixture values for `adc_max4095`, not measured board output or a claimed default ADC resolution.

Ring buffer / latest four samples



WHAT THE DATA SHOWS

After the fifth push the oldest 10 is overwritten. Remaining 20, 30, 40, 50 average 35. This is a host calculation, not hardware timing.

What You Achieved

1

LO1

Determine basic software components according to functional and business specifications (K1/A1).

2

LO2

Apply software design methodologies and tools in accordance with organisational practices (K2/A2).

3

LO3

Select controls, elements and features to meet design objectives (K3/A3).

4

LO4

Examine functionality and interoperability of software components (K4/A4).

5

LO5

Generate detailed design documentation mapped to user specifications (K5/A5).

Download course material on the LMS

<https://lms-tms.tertiaryinfotech.com/>

My course

AI-Assisted C Programming for Arduino

Course material

Slides / Learner Guide / Activities

Assessment

Written Assessment / Practical Performance

Course-specific access

Use your own learner account and the verified course title.

Course material and assessment areas are distinct. Trainer answers are not learner downloads.

Interface schematic; labels are illustrative, not a live authenticated screenshot.

[Open learner LMS](https://lms-tms.tertiaryinfotech.com/)

Assessment / final reminder

Written Assessment

- Day2 4:00-5:00 PM
- 60 minutes /5 questions
- K1-K5 / open book

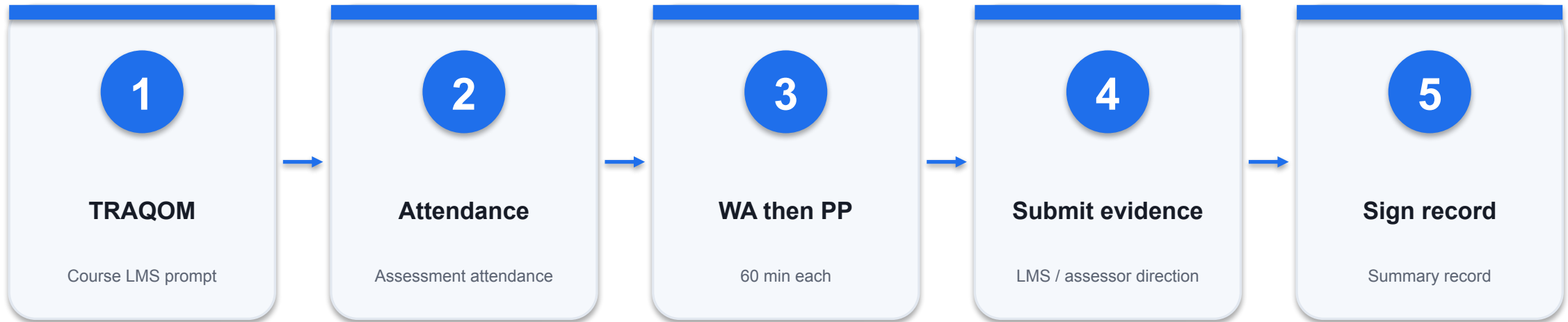
Practical Performance

- Day2 5:00-6:00 PM
- 60 minutes /5 tasks
- A1-A5 / individually

Submission & competence

- Follow issued paper instructions
- All questions/tasks attempted
- Assessor confirms competence

Assessment Flow



ACTUAL PAPERS

Five WA questions K1-K5 and five PP tasks A1-A5. Follow the assessor-issued submission instructions.

Digital Attendance (Mandatory)

1

TRAQOM / SSG

Use the course-specific digital attendance prompt supplied on the LMS.

2

Assessment attendance

Complete the separate assessment attendance record when directed.

3

Your identity

Use only your own learner record; do not share sign-in details.

4

Trainer support

Ask the trainer if the course/session identity does not match.

TERTIARY INFOTECH ACADEMY

Thank You

Keep specifications, C contracts, tests and user documentation aligned.
enquiry@tertiaryinfotech.com / +65 6100 0613